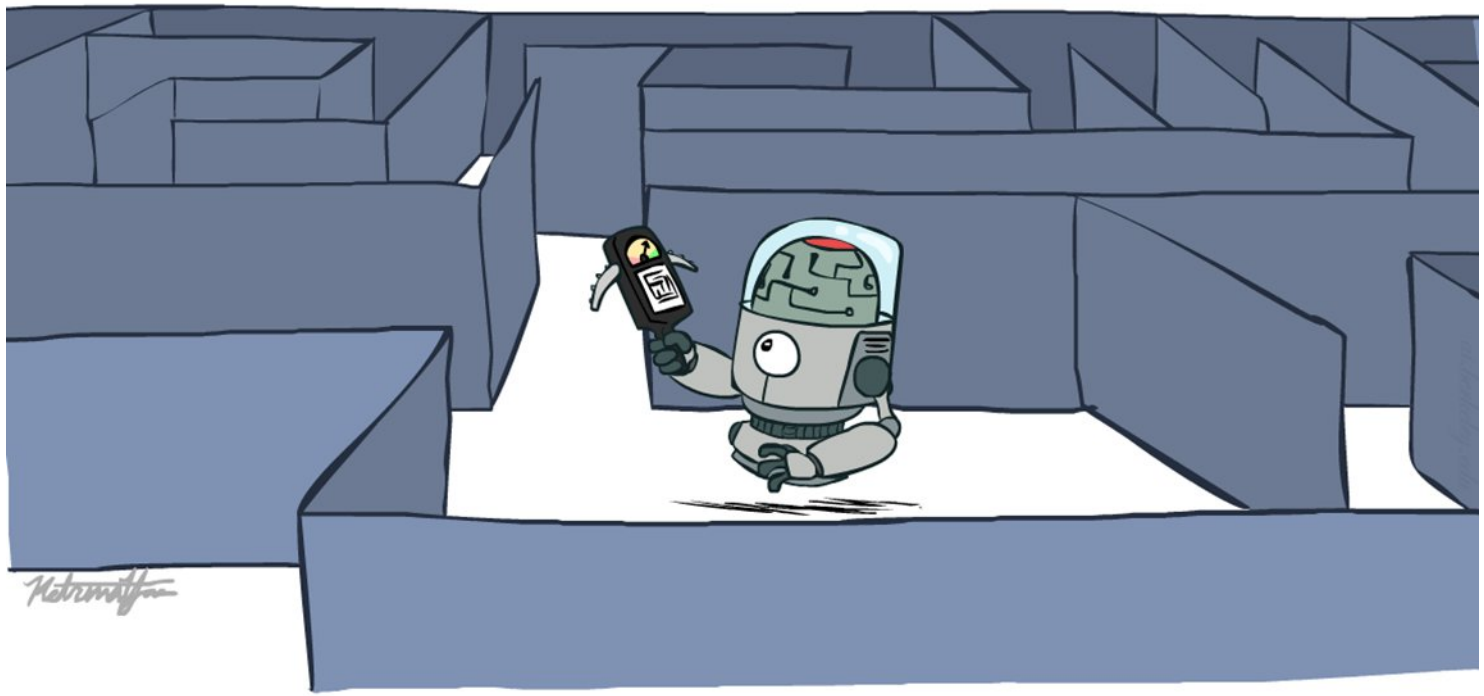
Informed Search

Heuristics Search



Uninformed Search Vs. Informed Search

- **<u>Uninformed Search:</u>**
  - “Which node should I explore next?” no estimate of closeness to the goal

- **<u>Informed Search:</u>**
  - “Which node looks more promising?” uses a heuristic to make that judgement

Informed Search

- If you need to reach a destination, you *could* systematically explore roads, but information about estimated distance to the destination lets you prioritize roads that appear to take you toward it.

- In informed search, the additional information used to guide the search is generally represented by a **heuristic function**. That's why informed search is also commonly called heuristic search.

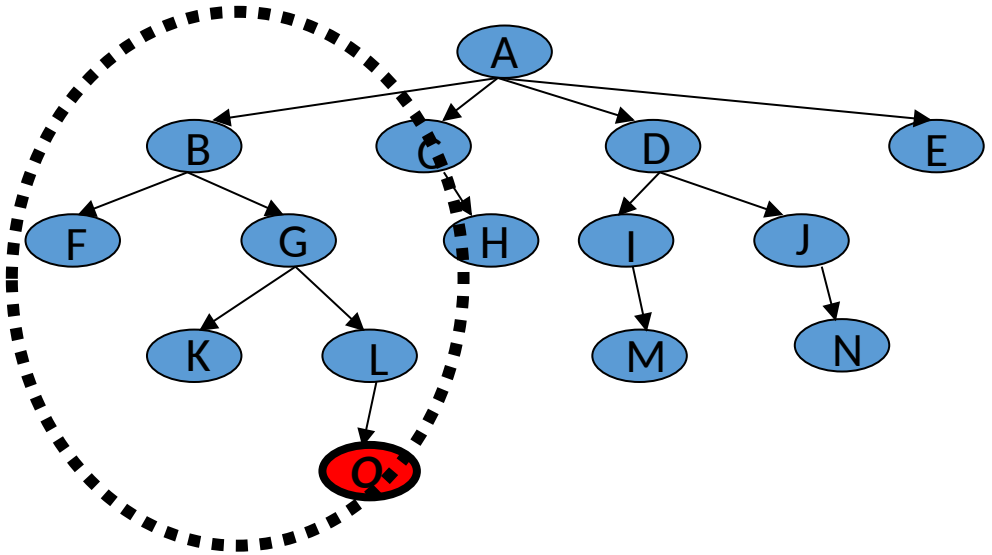
Today

- Informed Search
  - Heuristics
  - Greedy Search
  - A* Search

Using problem specific knowledge to aid searching

- With knowledge, one can search the state space as if he was given "hints" when exploring a maze.
  - Heuristic information in search = Hints
  - Imagine solving a maze.
    Uninformed search = randomly exploring paths.
    Informed search = someone telling you "the exit is somewhere on the right side".
- Leads to dramatic speed up in efficiency.

Search only in this sub-tree!!



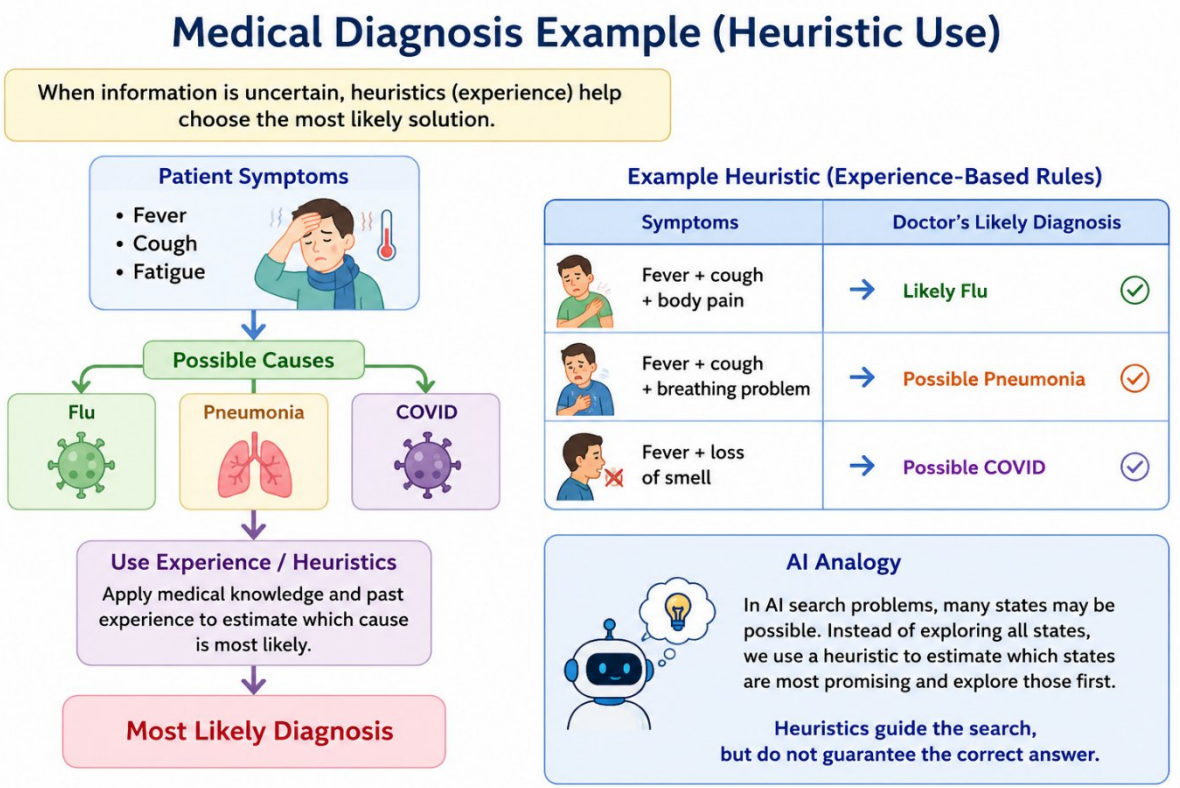
What are Heuristics?

- A heuristic is an estimate that helps the search algorithm determine how promising a state is with respect to reaching the goal.
- Heuristics are obtained through a lot of experiences.
- Heuristics often enable us to make decisions quickly without thinking deeply about the reasons.
- The more experiences we have, the better the heuristics will be.
- Example in real life:
  - Doctors diagnosing disease based on symptoms
  - Experienced chess players predicting good moves
- Heuristics **do not guarantee** **the best solution**, but they **guide the search effectively**.

When Heuristics are used?

AI problem solvers employ heuristics in two basic situations:

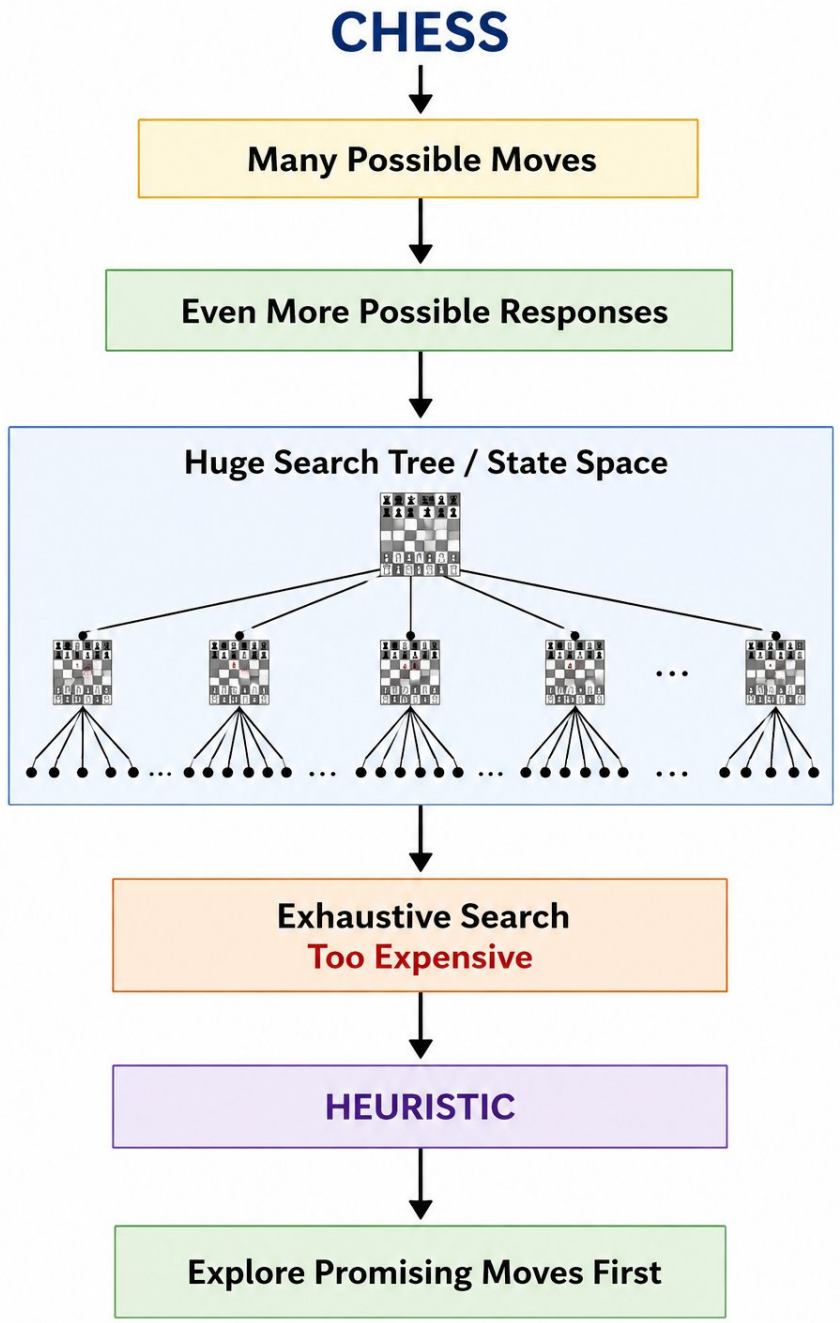
1. When there is uncertainty – when the available information **does not clearly identify** the correct choice



When Heuristics are used?

2. A problem may have an exact solution, but the **computational cost of finding it may be prohibitive**.

- The solution exists, but searching every possibility would take too much time or memory.
- Chess: state space growth is combinatorically explosive, with the number of possible states increasing with the depth of the search.
- Heuristics attack this complexity by guiding the search along the most "promising" path through the space.

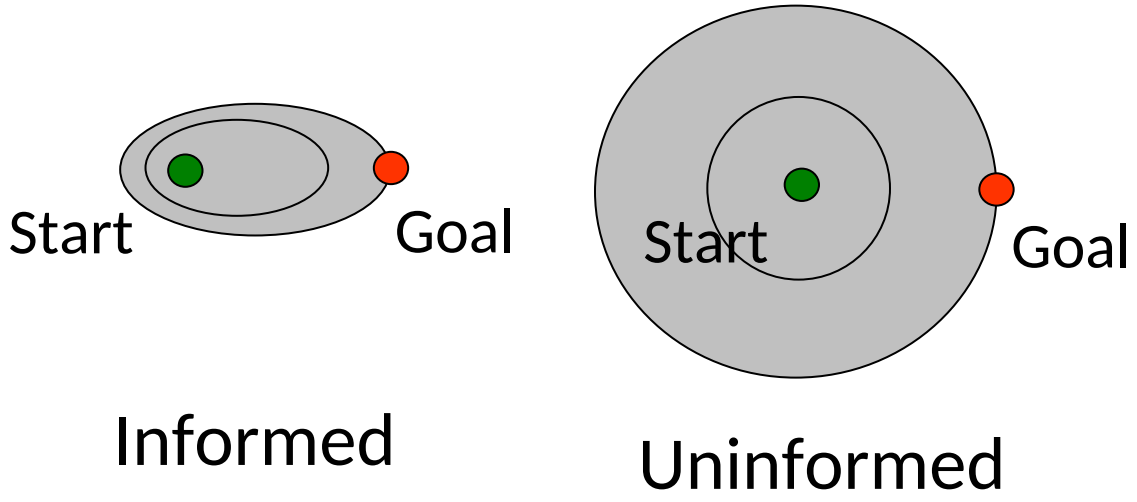


Why heuristic functions work?

- **Uninformed Search** : at most *b* choices at each node and a depth of *d* at the goal node
  - In the worst case, search around $O\left(b^d\right)$ nodes before finding a solution (Exponential Time Complexity).
  - explores **many unnecessary nodes**

- **Heuristics** improve the efficiency of search algorithms by reducing the effective branching factor from *b* to (ideally) a lower constant b^* such that $1 \leq b^* \ll b$
  - explores **nodes that seem closer to the goal**.

- Guide search ***towards the goal*** instead of ***all over the place***



Evaluation Function

An **evaluation function** gives an estimation on the **"cost" (or desirability)** of a node in a tree/graph.



Purpose

It helps the search algorithm decide which node to expand next.

We select the node with the **best (lowest) evaluation** among all choices.

Example

Frontier (Open List)	Evaluation f(n)	Interpretation
A	12	Less desirable
B	5	Most desirable ✓
C	9	Moderately desirable

Expand node **B** because it has the lowest cost (most desirable).

Connection to Search Algorithms



Why do we need it?

When several nodes are in the frontier, the evaluation function provides a rule to rank and prioritize them.

How it is used

Nodes are kept in the frontier (often a priority queue) sorted by their evaluation (lowest first).

Priority Queue (Frontier)

B	5	C	9	A	12	...

Expand from left (lowest cost first)



Key Takeaway

The evaluation function is the "decision making rule" that guides the search toward the most promising nodes.

Different search algorithms use different evaluation functions.

Evaluation / Heuristic Function

Suppose we are travelling from Lahore to Islamabad and we are currently at some intermediate city. There are two things we might want to know: how much distance have we already travelled, and approximately how much distance is still left?

Evaluation / Heuristic Function

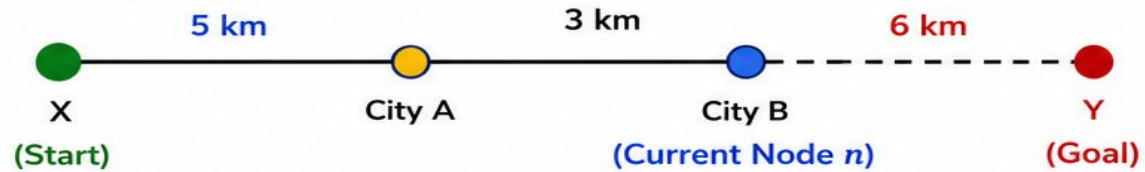
- Evaluation function $f(n) = g(n) + h(n)$
  - $g(n)$ = exact cost to reach n
  - $h(n)$ = estimated cost to goal from n
  - $f(n)$ = estimated total cost of cheapest path through n to goal

Example: Evaluating a Node

Suppose we are travelling from X . to Y . and we are currently at City B.
We want to know:

1. How much distance have we already travelled?
2. Approximately how much distance is still left?

Path and Costs



Step 1: Compute $g(n)$

$g(n)$ = exact cost from Start to n

Cost from X to City B = 5 + 3 = 8 km

So, $g(B) = 8$ km

Step 2: Estimate $h(n)$

$h(n)$ = estimated cost from n to Goal

Estimated distance from City B to Y = 6 km

So, $h(B) = 6$ km

Step 3: Compute $f(n)$

$f(n) = g(n) + h(n)$

$f(B) = 8 + 6 = 14$ km

So, $f(B) = 14$ km

Summary at City B (n)

Component	Meaning	Value
$g(n)$	Cost so far (Start → n)	8 km
$h(n)$	Estimated cost (n → Goal)	6 km
$f(n) = g(n) + h(n)$	Estimated total cost (cheapest path via n to Goal)	14 km

Key Takeaway

- $g(n)$ is the cost we have already paid.
- $h(n)$ is our best guess of what remains.
- $f(n)$ is our best guess of the total cost.

At City B: $g(B) = 8$ km, $h(B) = 6$ km, and $f(B) = 14$ km.

Special Cases of the Evaluation Function

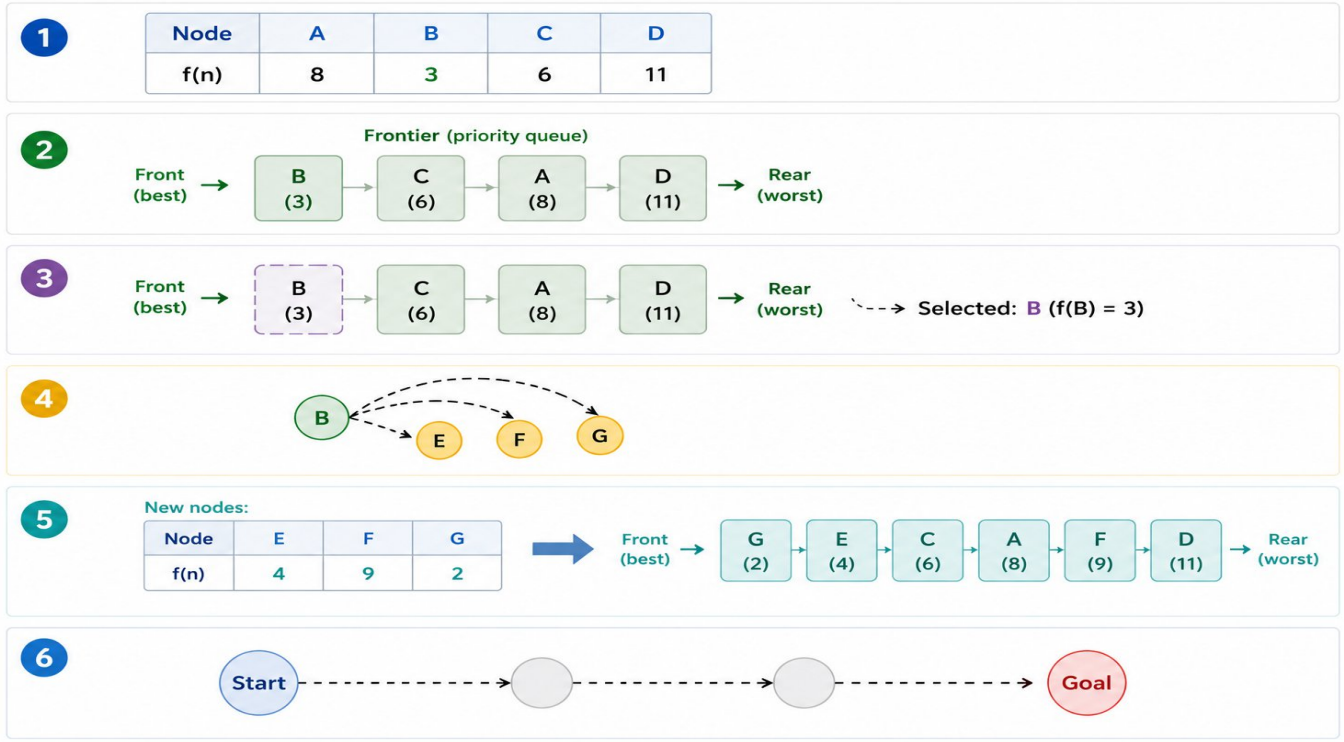
Different search strategies use different parts of the evaluation function $f(n) = g(n) + h(n)$ to decide which node to expand next.

Search Strategy	Evaluation Function $f(n)$	What It Considers	What It Ignores	How It Chooses Next Node
1) Uniform Cost Search (UCS)	$f(n) = g(n)$ ($h(n)$ is not used)	Cost so far from Start to n $g(n)$	Estimated cost from n to Goal $h(n)$	Expands the node with the **smallest cost so far** ($g(n)$).
Example: If $g(A)=7$, $g(B)=4$, $g(C)=10$ ⇒ UCS expands B (because 4 is smallest).				
2) Greedy (Best-First) Search	$f(n) = h(n)$ ($g(n)$ is not used)	Estimated cost from n to Goal $h(n)$	Cost so far from Start to n $g(n)$	Expands the node that appears **closest to the goal** (**smallest $h(n)$**).
Example: If $h(A)=8$, $h(B)=12$, $h(C)=3$ ⇒ Greedy expands C (because 3 is smallest).				
3) A* Search	$f(n) = g(n) + h(n)$ (uses both $g(n)$ and $h(n)$)	Both cost so far and estimated cost $g(n)$ and $h(n)$	**Nothing (uses both)**	Expands the node with the **lowest estimated total cost** $g(n) + h(n)$.
Example: If $g(A)=4$, $h(A)=8$ ⇒ $f(A)=12$; $g(B)=7$, $h(B)=3$ ⇒ $f(B)=10$; $g(C)=2$, $h(C)=10$ ⇒ $f(C)=12$ ⇒ A* expands B (because 10 is smallest).				

Key Takeaway: UCS looks **backward** (actual cost), Greedy looks **forward** (estimated cost), and A* looks **both backward and forward** to make a balanced decision.

How it works?

1. Evaluate
2. Store in frontier
3. Select
4. Expand
5. Evaluate new nodes
6. Repeat step 3-5



The evaluation function determines what “most promising” means.

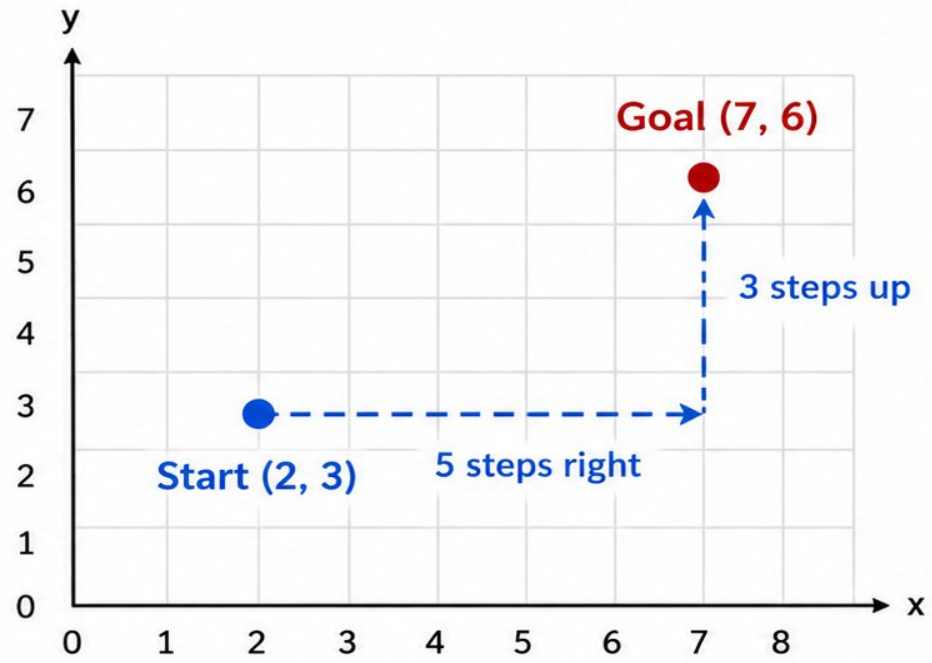
Heuristic Function

- A heuristic function *h(n)* estimates the cost or distance from the current state *n* to the goal.

- It is an **estimate**, not the actual cost.
- It is **problem-specific**.
- A good heuristic helps search focus on promising states.

- **Examples:** Manhattan Distance and Euclidean Distance

Manhattan Distance



- **Manhattan distance** measures distance if you can only move in **four directions**:



- **No** diagonal movement is allowed.

- Formula:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

  - (x_1, y_1) = current position
  - (x_2, y_2) = goal position

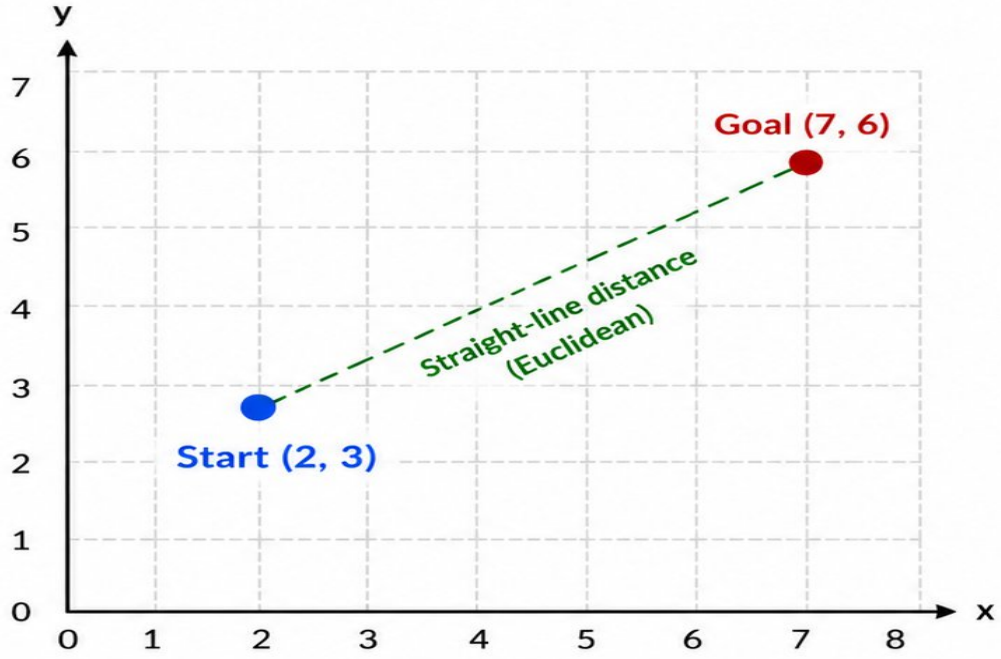
Example:

Start (2, 3) → **Goal (7, 6)**

$$h(n) = |7 - 2| + |6 - 3|$$
$$= 5 + 3$$
$$= \boxed{8}$$

Heuristic	Used When
Manhattan Distance	Movement only in grid directions (like 8-Puzzle)

Euclidean Distance



- **Euclidean distance** measures the **straight-line distance** between two points.
- It is used as a **heuristic estimate** of the remaining cost when movement is allowed in **any direction** (including diagonals).

- Formula:

$$h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

- (x_1, y_1) = current position
- (x_2, y_2) = goal position

Example:

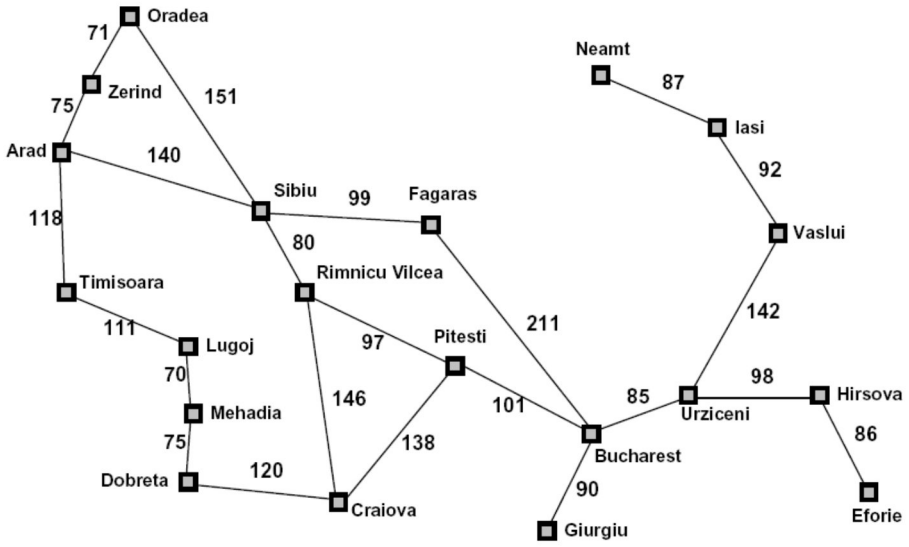
Start (2, 3) → **Goal (7, 6)**

$$h(n) = \sqrt{(7-2)^2 + (6-3)^2}$$
$$= \sqrt{(5)^2 + (3)^2}$$
$$= \sqrt{25 + 9}$$
$$= \sqrt{34}$$
$$\approx 5.83$$

Aspect	Euclidean Distance
Movement Allowed	Any direction (including diagonals)
Path Assumed	Straight-line path
Distance Computed	Straight-line distance between points
Use When	Movement allowed in any direction (e.g., robot in open space, continuous grids)
Note	It is a heuristic estimate and may be less than or equal to the actual path cost.

Example: Heuristic Function

Our goal is Bucharest. For every city x, we have a heuristic value h(x), which estimates how far that city is from Bucharest.



Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

h(x)

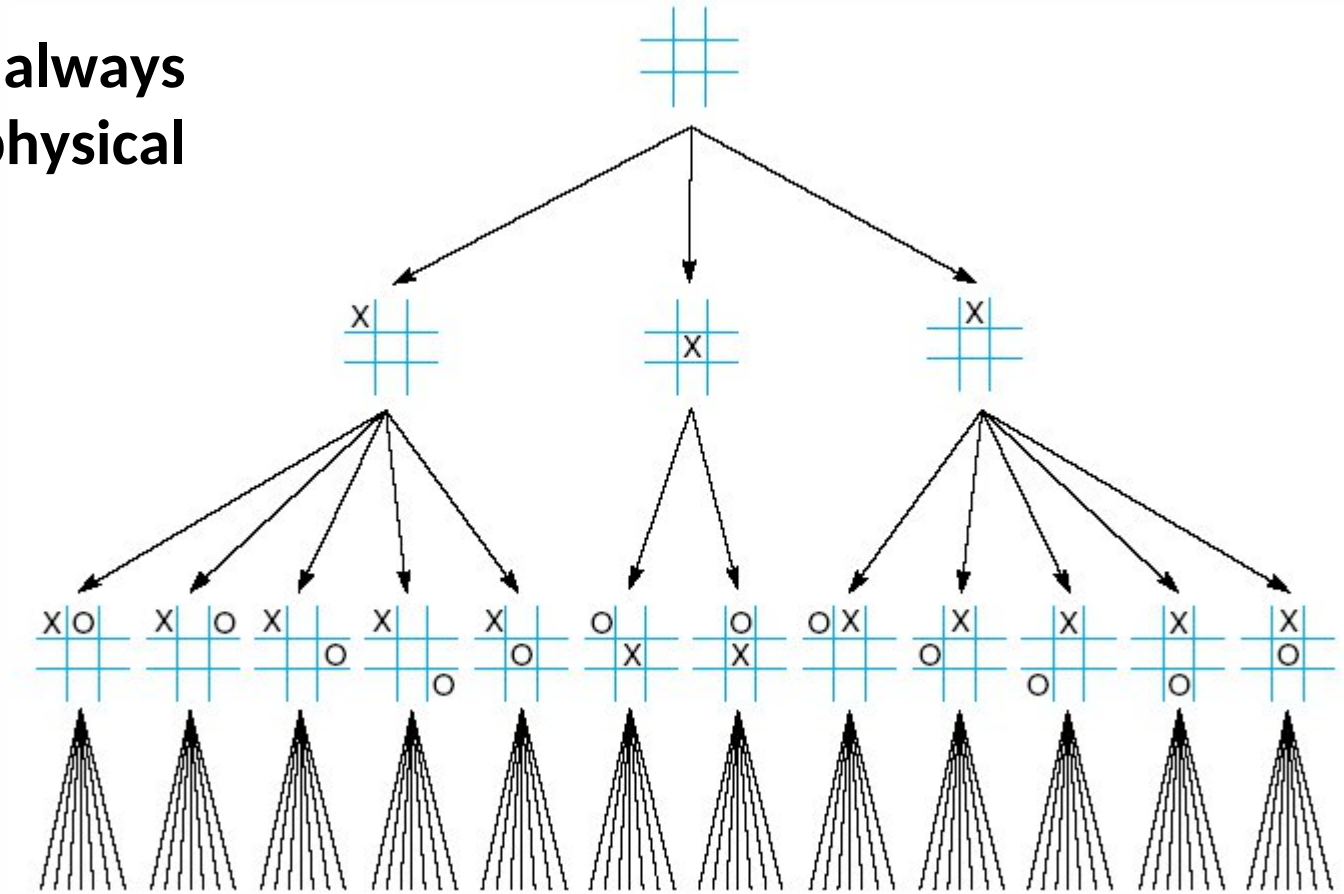
The red table shows the **heuristic function values**.
These are **straight-line distances from each city to Bucharest**.

h(x) = estimated distance from city x to the goal (Bucharest)

h(Arad) = 366 straight line distance from Arad to Bucharest is 366

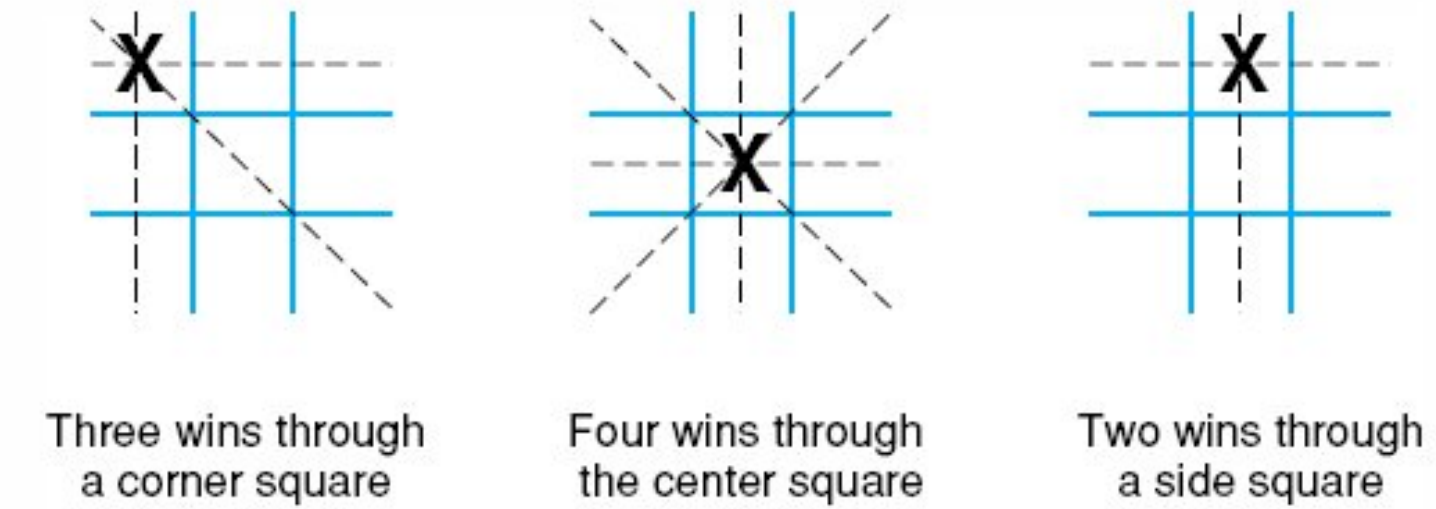
First three levels of the tic-tac-toe state space reduced by symmetry

A heuristic does not always have to be a physical distance.



Rotating the board produces the **same state**, so we ignore duplicates.

The “most wins” heuristic applied to the first children in tic-tac-toe.

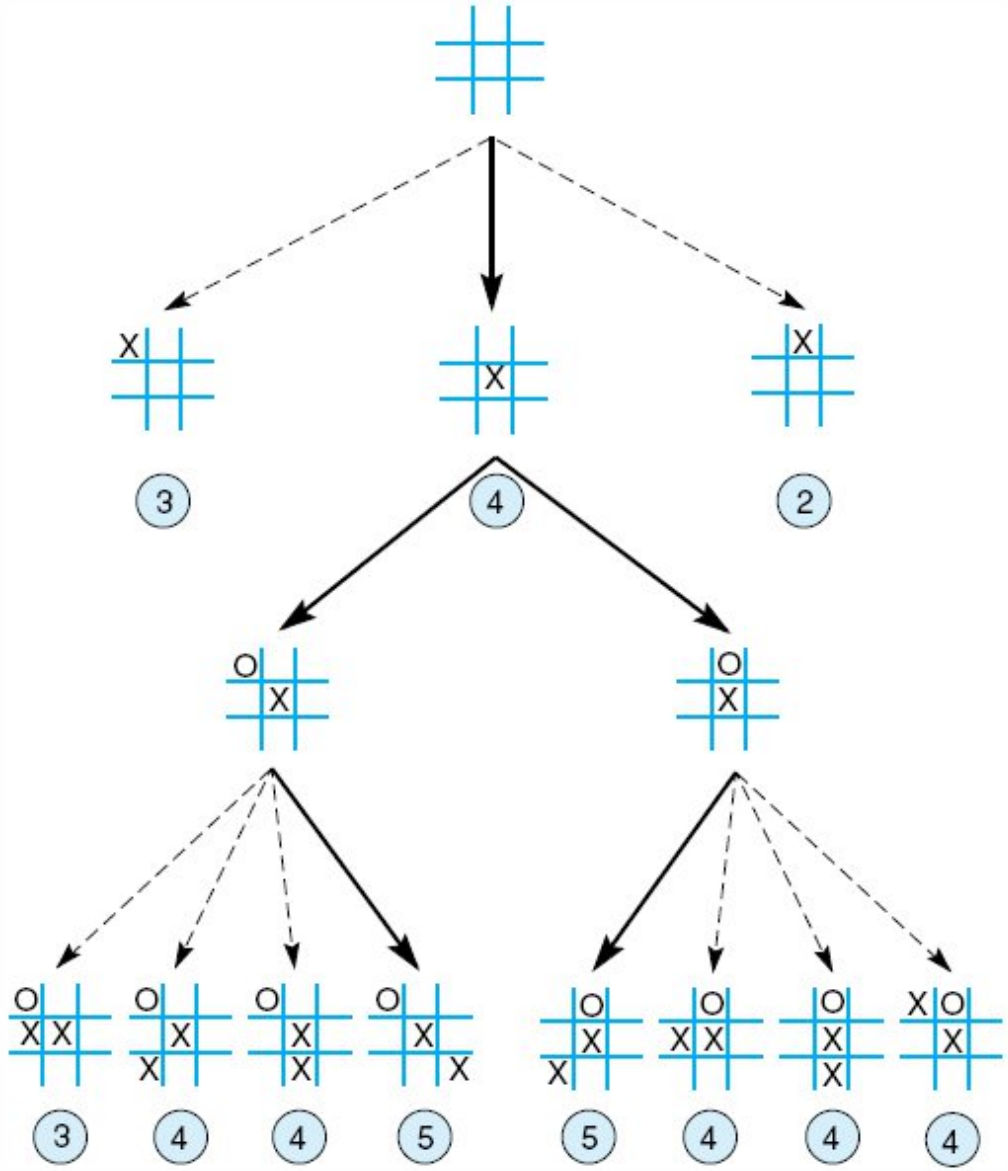


<u>Most Wins Heuristic</u>: Choose moves that lead to **maximum winning possibilities**.

Heuristically reduced state space for tic-tac-toe

- **The Top Node (Initial State)**
- At the top, we see an **empty board**.
  - From this state, player **X** can place a mark in **any of the 9 squares**.
  - But many moves are **symmetrical** (they lead to equivalent boards).
  - So instead of exploring all 9 possibilities, we consider only **3 representative moves**:
    1. Corner
    2. Center
    3. Edge

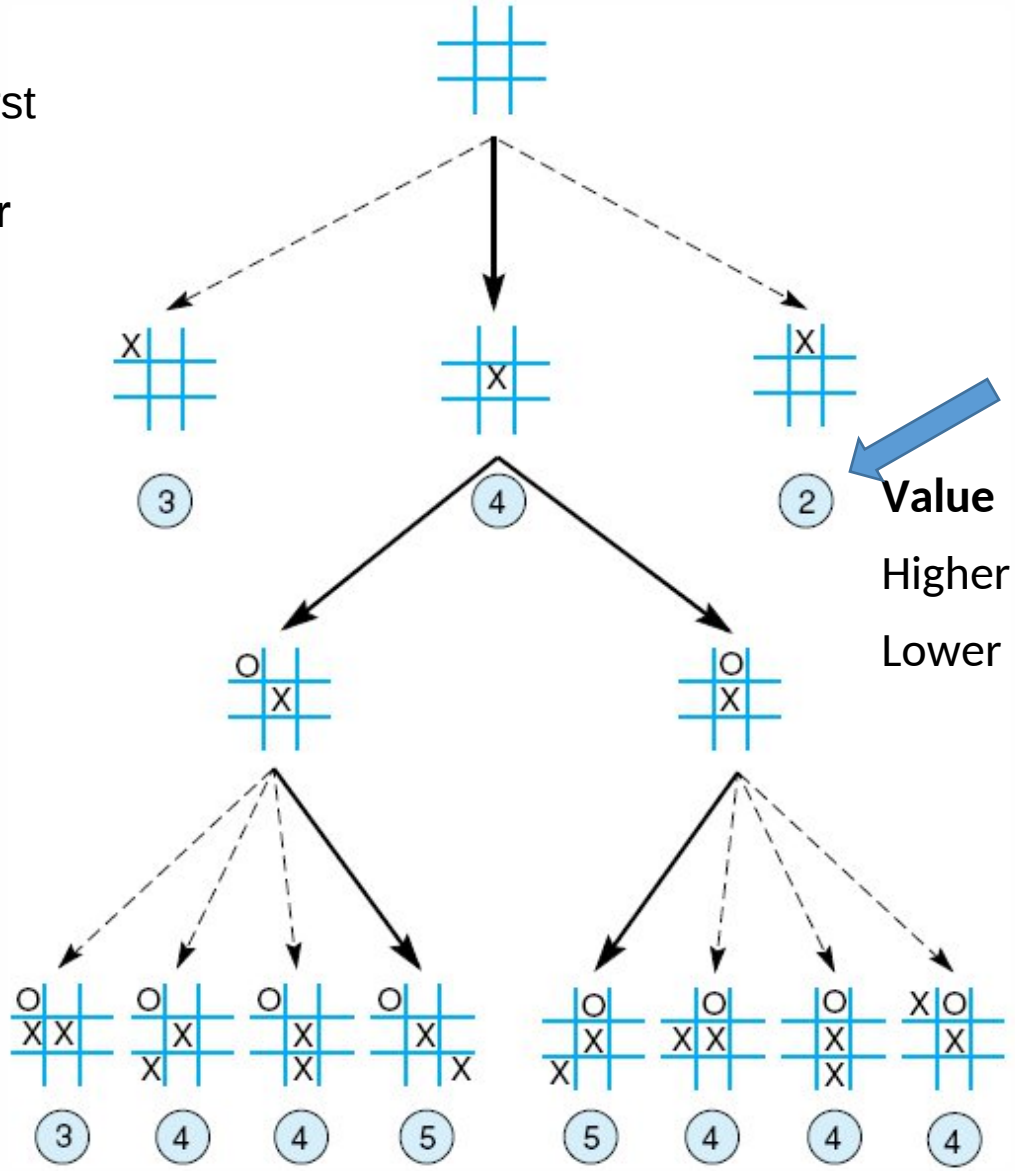
This already **reduces the search space**.



Heuristically reduced state space for tic-tac-toe.

First Level of the Tree

- The diagram shows three possible first moves for **X**.
- Under each board you see a **number (2, 3, 4, etc.)**.
- These numbers represent the **heuristic value**.
  - Meaning: How promising this board state is for player X.



Value	Meaning
Higher number	Better position for X
Lower number	Worse position

Heuristically reduced state space for tic-tac-toe.

Choosing the Best Move

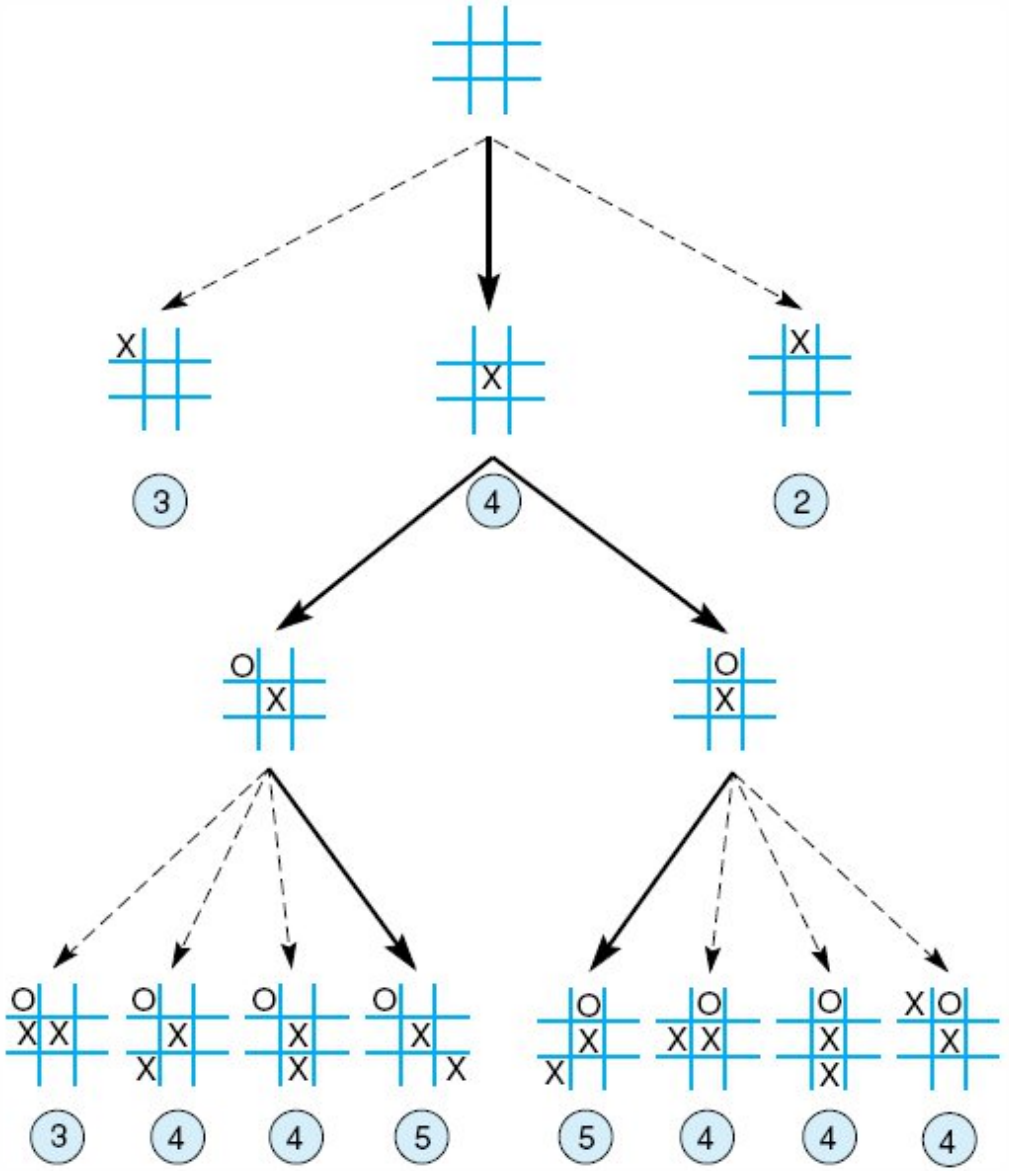
Suppose the heuristic values are:

The algorithm chooses:

Middle board (value = 4)

- Because the heuristic says this move gives **better chances to win**.

This means the algorithm **does not explore all branches equally**.

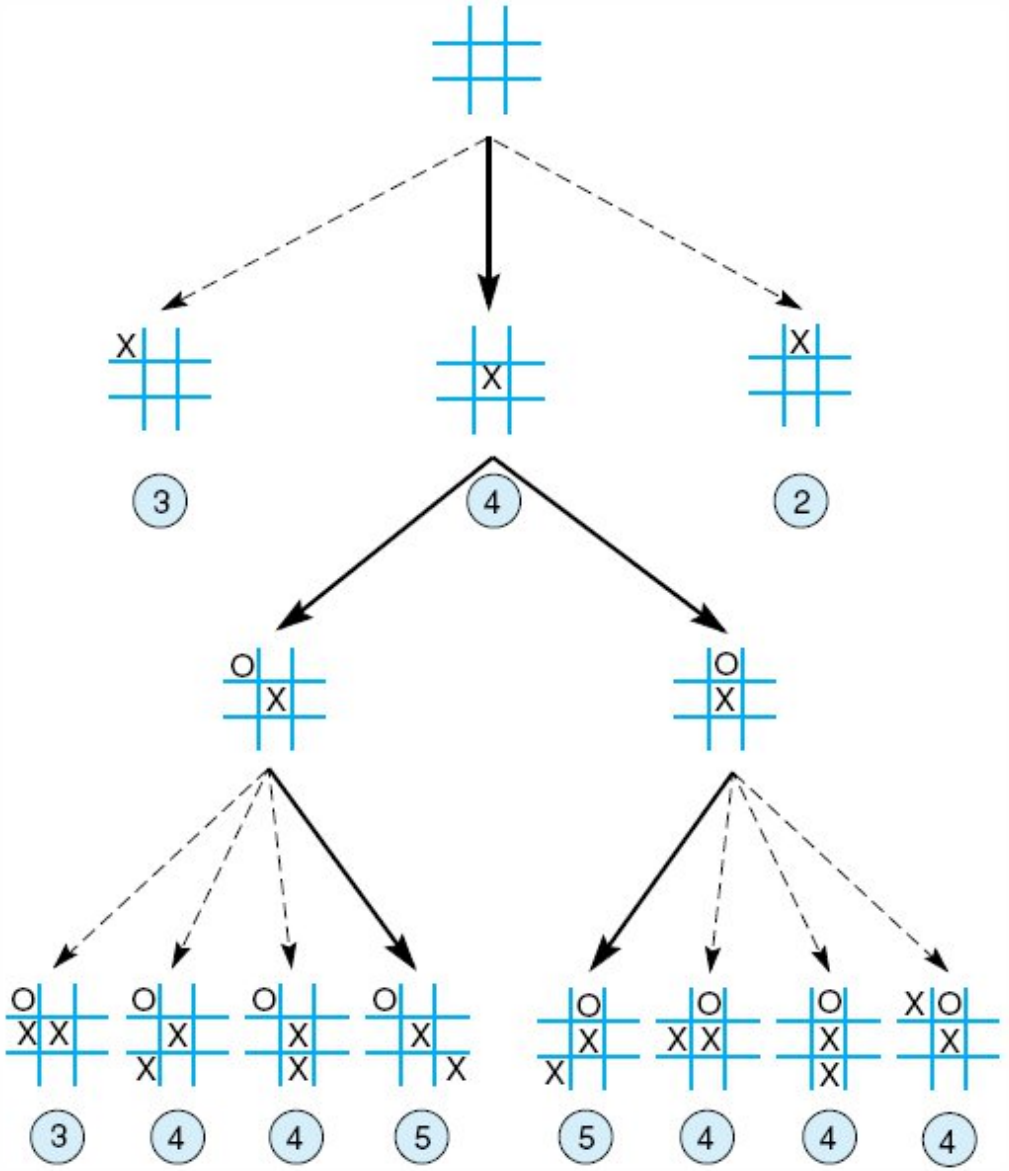


Heuristically reduced state space for tic-tac-toe.

Second Level of the Tree

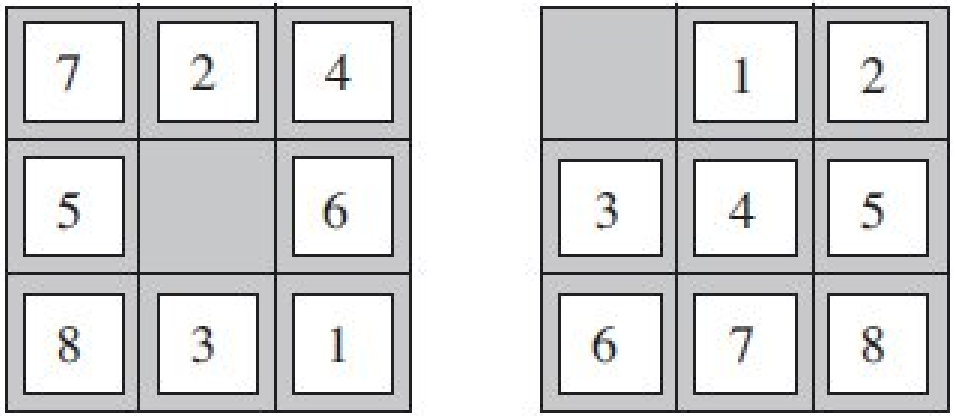
- Now player **O** makes a move.
- From the chosen board, multiple states appear again.
- Each resulting board again receives a **heuristic evaluation**.
- Examples in the diagram:
  - 3
  - 4
  - 5

The algorithm again prefers the **highest heuristic value**.



Instead of checking every possible Tic-Tac-Toe board, the AI evaluates each board using a heuristic score and only explores the most promising moves.

8–Puzzle: Heuristics?



One problem can have multiple possible heuristic functions.

- h1: The number of misplaced tiles (squares with number).
- h2: The sum of the distances of the tiles from their goal positions.

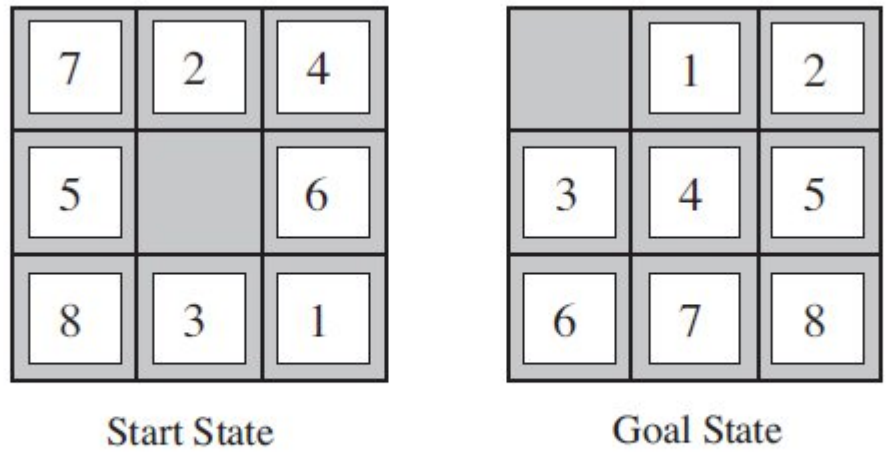
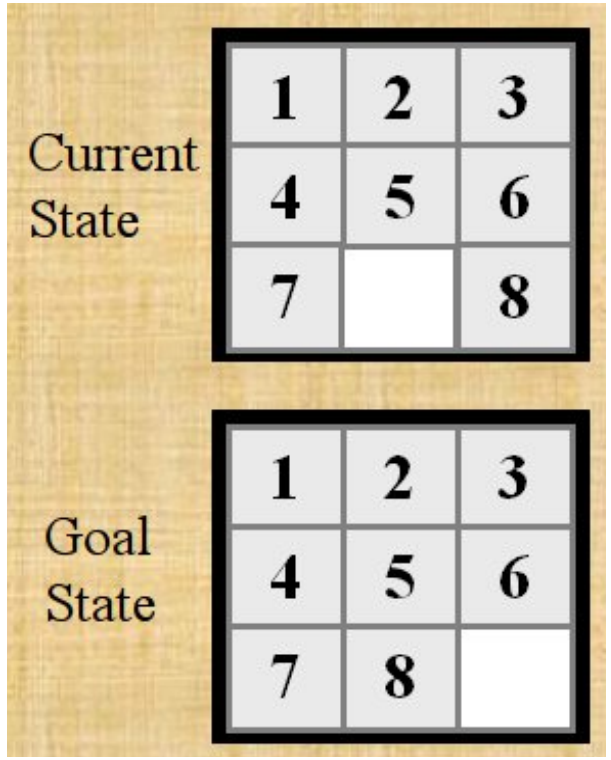


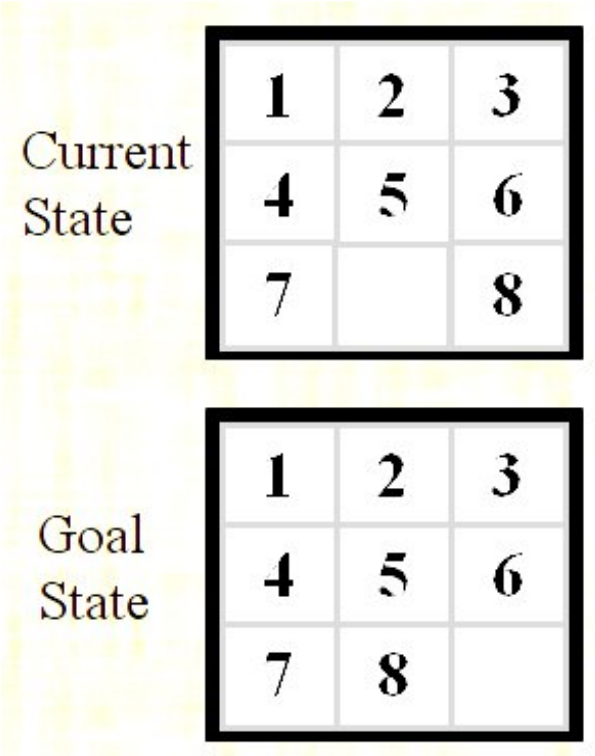
Figure 3.28 A typical instance of the 8-puzzle. The solution is 26 steps long.

- h_1 = the number of misplaced tiles. For Figure 3.28, all of the eight tiles are out of position, so the start state would have $h_1 = 8$. h_1 is an admissible heuristic because it is clear that any tile that is out of place must be moved at least once.
- h_2 = the sum of the distances of the tiles from their goal positions. Because tiles cannot move along diagonals, the distance we will count is the sum of the horizontal and vertical distances. This is sometimes called the **city block distance** or **Manhattan distance**. h_2 is also admissible because all any move can do is move one tile one step closer to the goal. Tiles 1 to 8 in the start state give a Manhattan distance of

$$h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18 .$$


Current State
1 2 3
4 5 6
7 8
Goal State
1 2 3
4 5 6
7 8

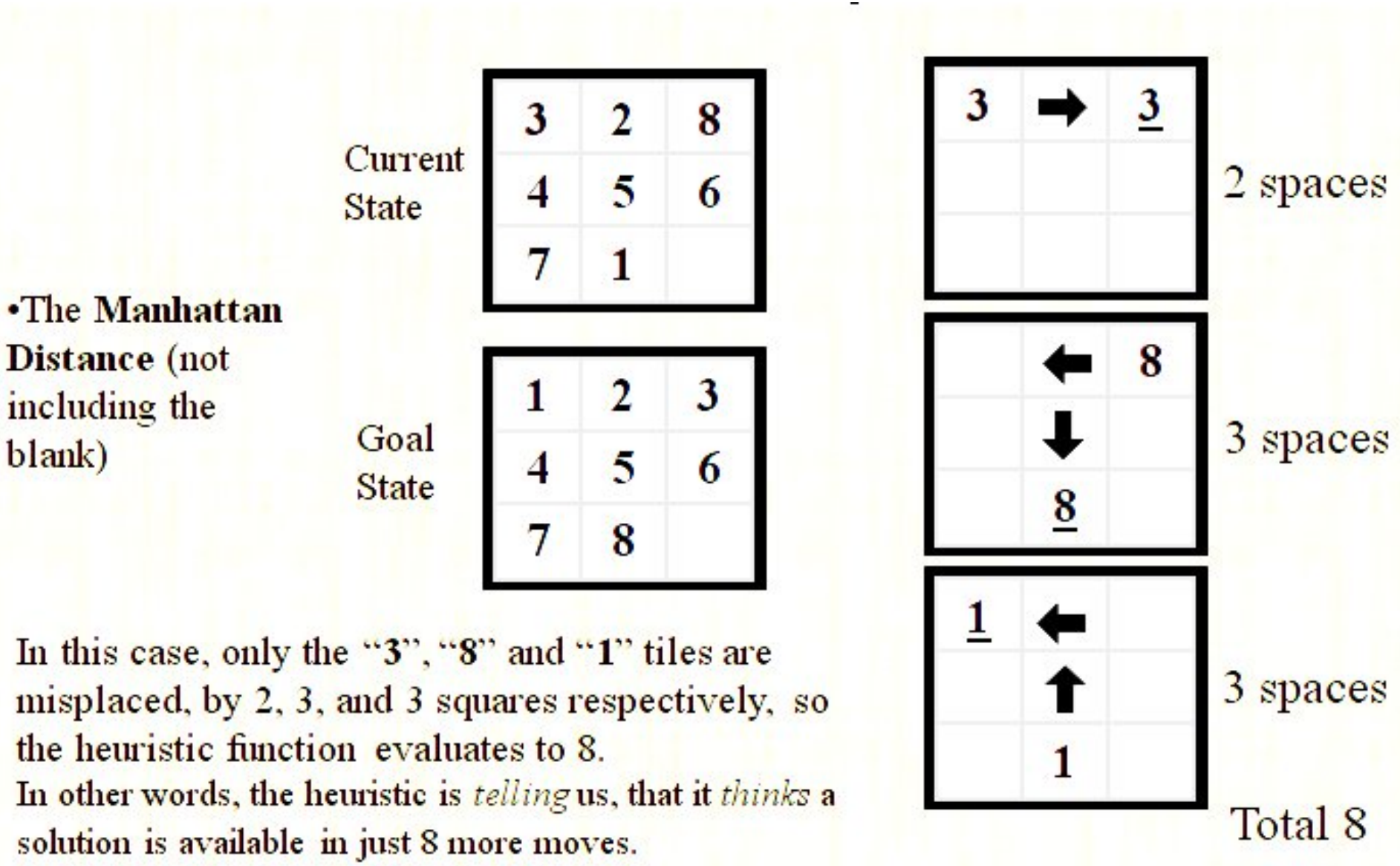
h1: The number of misplaced tiles (not including the blank)



- *Notation:* $f(n) = h(n)$
- *h(current state) = 1*
- Because this state has one misplaced tile.

- Only "**8**" is misplaced
- So the heuristic function evaluates to 1.
- The heuristic is *telling* us, that it *thinks* a solution might be available in just 1 more move.

h2: The sum of the distances of tiles from goal positions



Notation: $f(n) = h(n)$, *h(current state) = 8*



Greedy Best First Search

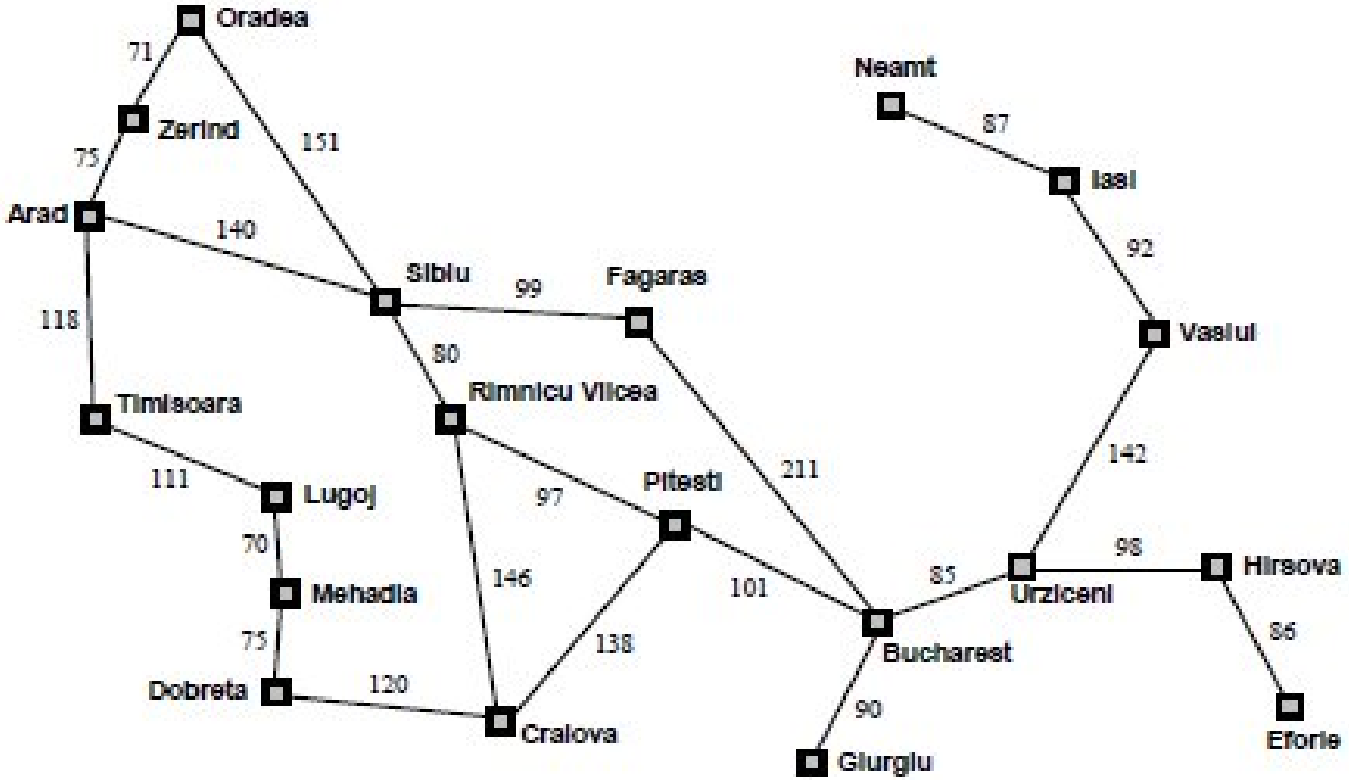
eval-fn: $f(n) = h(n)$

Greedy (Best-First) Search

- Greedy Best-First Search expands the node that appears closest to the goal.
- Use heuristic function f(n) = h(n)

- **Best-first search** tries to expand the node that is closest to the goal, on the grounds that this is likely to lead to a solution quickly.

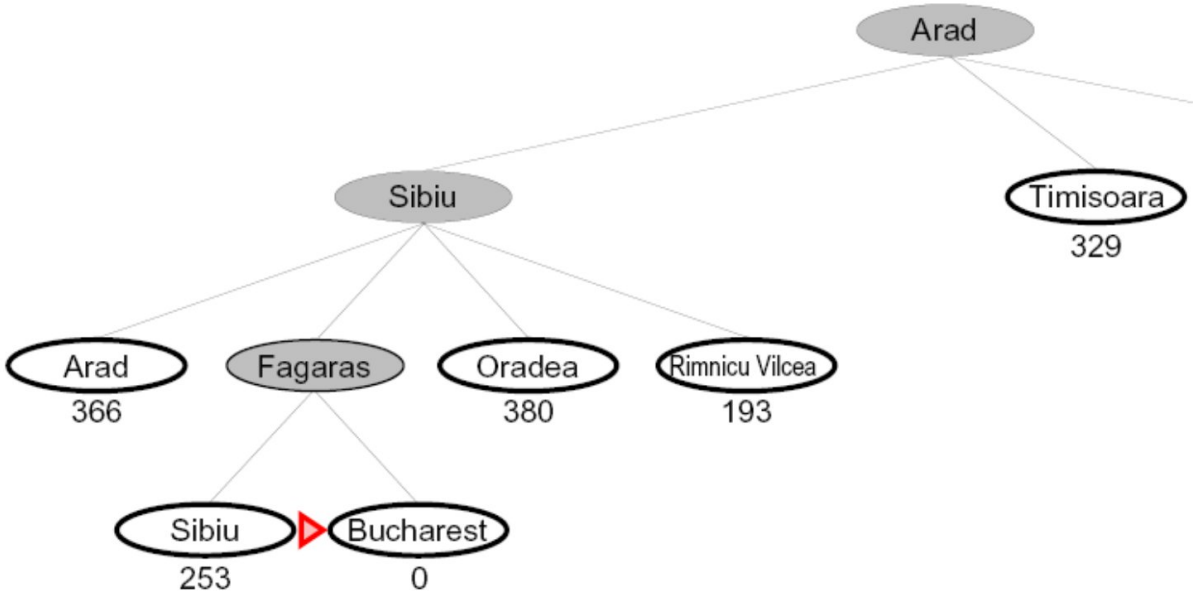
- *Choose path with low heuristic value*

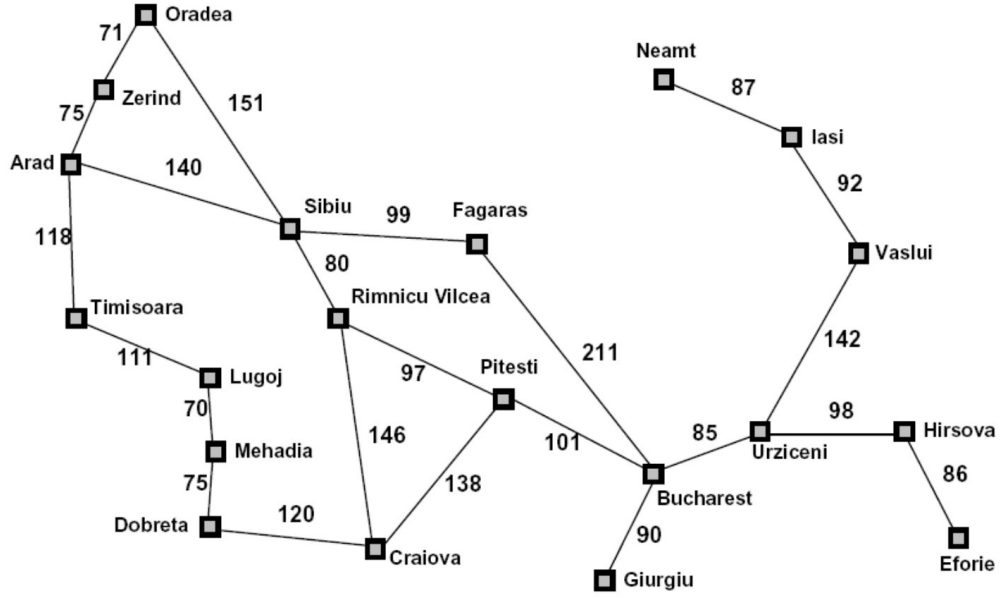


Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy Search

- Expand the node that seems closest…



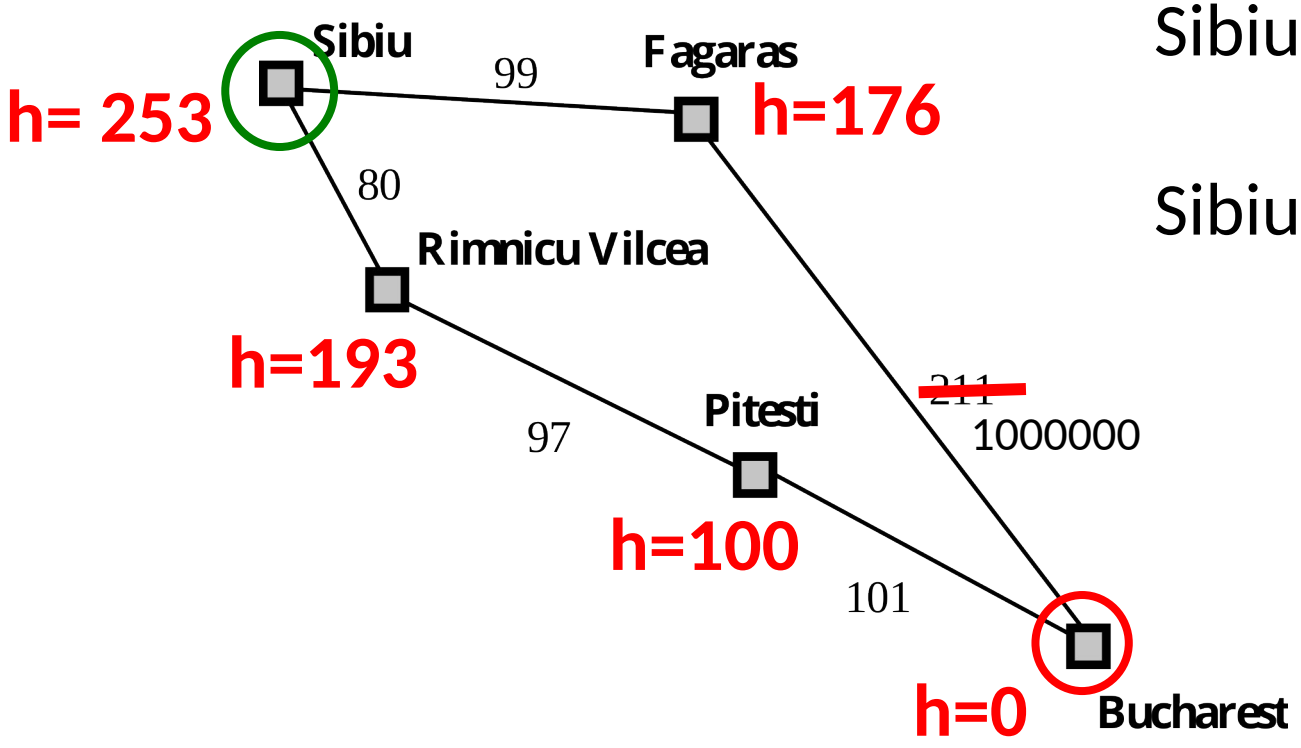


Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

- What can go wrong?

Greedy Search

- Expand the node that seems closest…(order frontier by h)
- What can possibly go wrong?

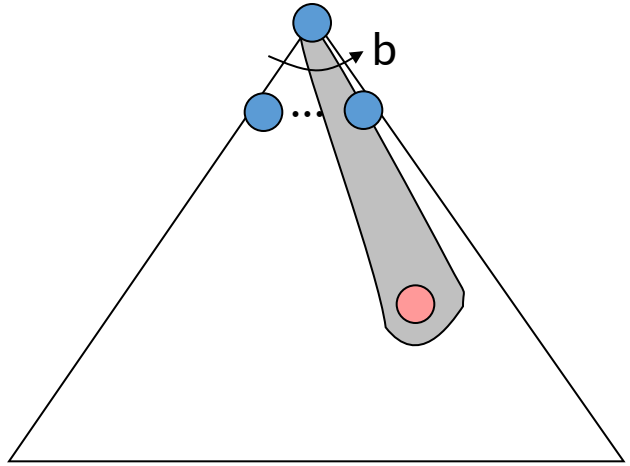


Sibiu-Fagaras-Bucharest =
99+211 = **310**

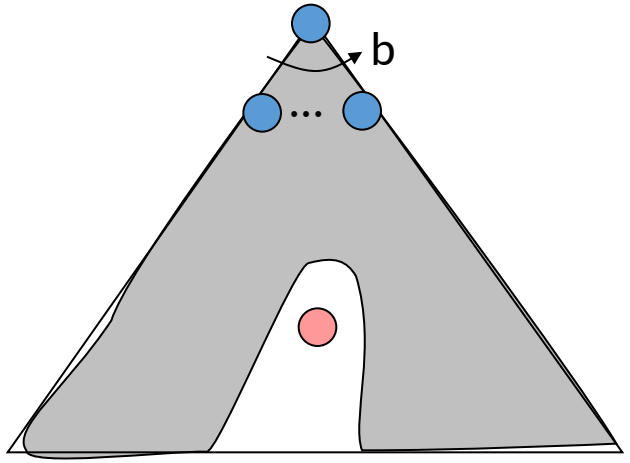
Sibiu-Rimnicu Vilcea-Pitesti-Bucharest =
80+97+101 = **278**

Greedy Search

- Strategy: expand a node that you think is closest to a goal state
  - Heuristic: estimate of distance to nearest goal for each state



- A common case:
  - Best-first takes you straight to the (wrong) goal



- Worst-case: like a badly-guided DFS

Best First Search (Greedy Search)

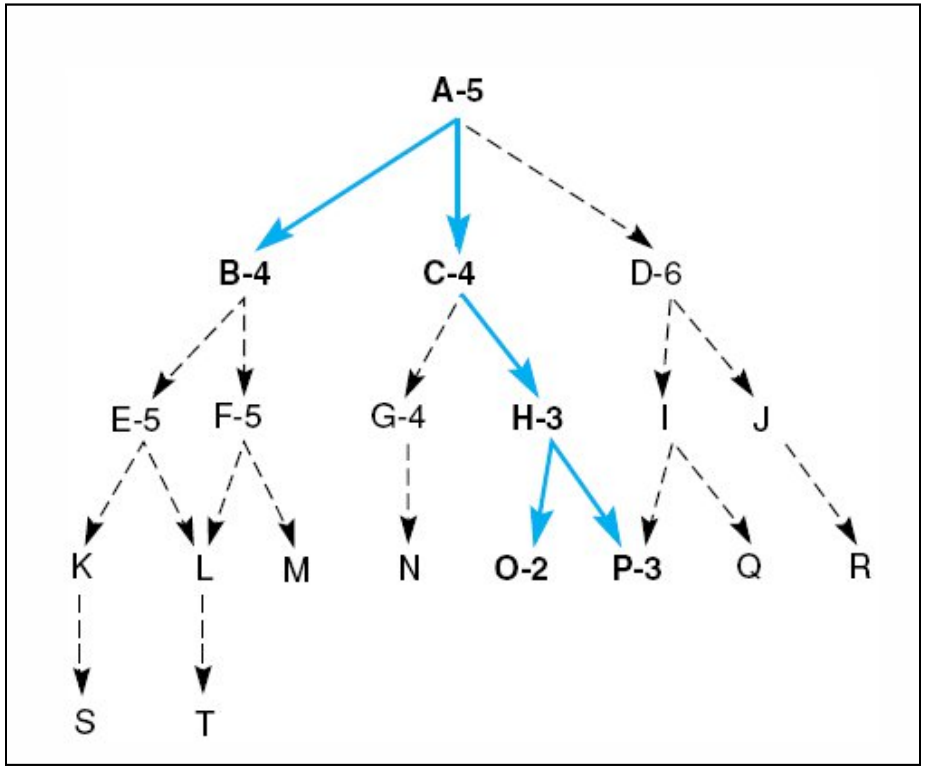
- Algorithm
- Let open be a priority queue containing initial state and closed be a queue containing visited states
  - Loop
  - If open is empty, return failure
  - else
    - Node Remove-First (open) //remove element with highest priority from open and place it in closed
      - If Node is goal then return path from initial to goal
    - else
      - Generate all successors of Node and place the successors into Open according to their heuristic values
  - End loop

```
function best_first_search;

begin
   open := [Start];                                                    % initialize
   closed := [ ];
   while open ≠ [ ] do                                                 % states remain
      begin
         remove the leftmost state from open, call it X;
         if X = goal then return the path from Start to X
         else begin
                  generate children of X;
                  for each child of X do
                  case
                        the child is not on open or closed:
                           begin
                                 assign the child a heuristic value;
                                 add the child to open
                           end;
                        the child is already on open:
                           if the child was reached by a shorter path
                           then give the state on open the shorter path
                        the child is already on closed:
                           if the child was reached by a shorter path then
                              begin
                                 remove the state from closed;
                                 add the child to open
                              end;
                  end;                                                 % case
                  put X on closed;
                  re-order states on open by heuristic merit (best leftmost)
               end;
return FAIL                                                            % open is empty
end.
```

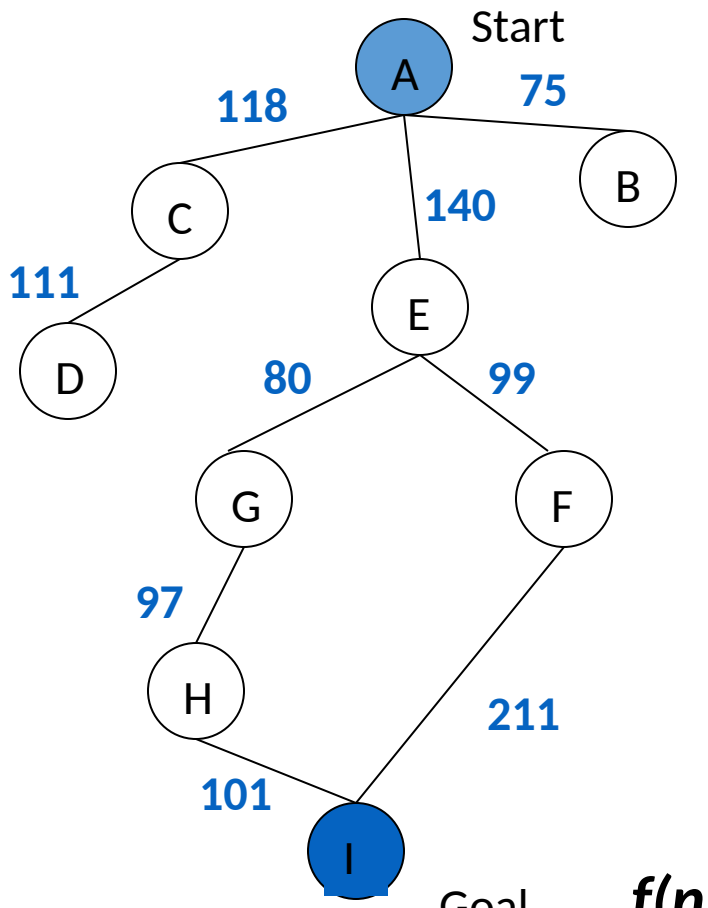
Best First Search

Task: Reach the goal state P



- Open = [A5]; closed = [ ]
- Evaluate A5; open = [B4, C4, D6]; closed = [A5]
- Evaluate B4; open = [C4, E5, F5, D6]; closed = [B4, A5]
- Evaluate C4; open = [H3, G4, E5, F5, D6]; closed [C4, B4, A5]
- Evaluate H3; open = [O2, P3, G4, E5, F5, D6]; closed [H3, C4, B4, A5]
- Evaluate O2; open = [P3, G4, E5, F5, D6]; closed [O2, H3, C4, B4, A5]
- Evaluate P3; the solution is found

Greedy Search



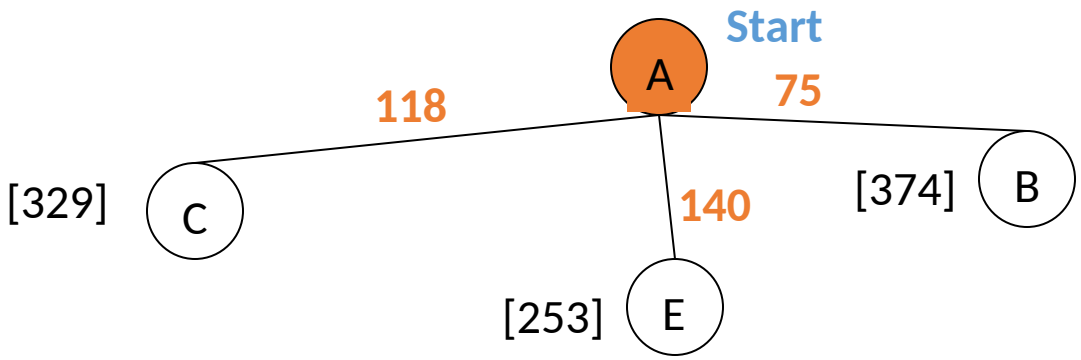
State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	**0**

***f(n) = h (n)* = straight-line distance heuristic**

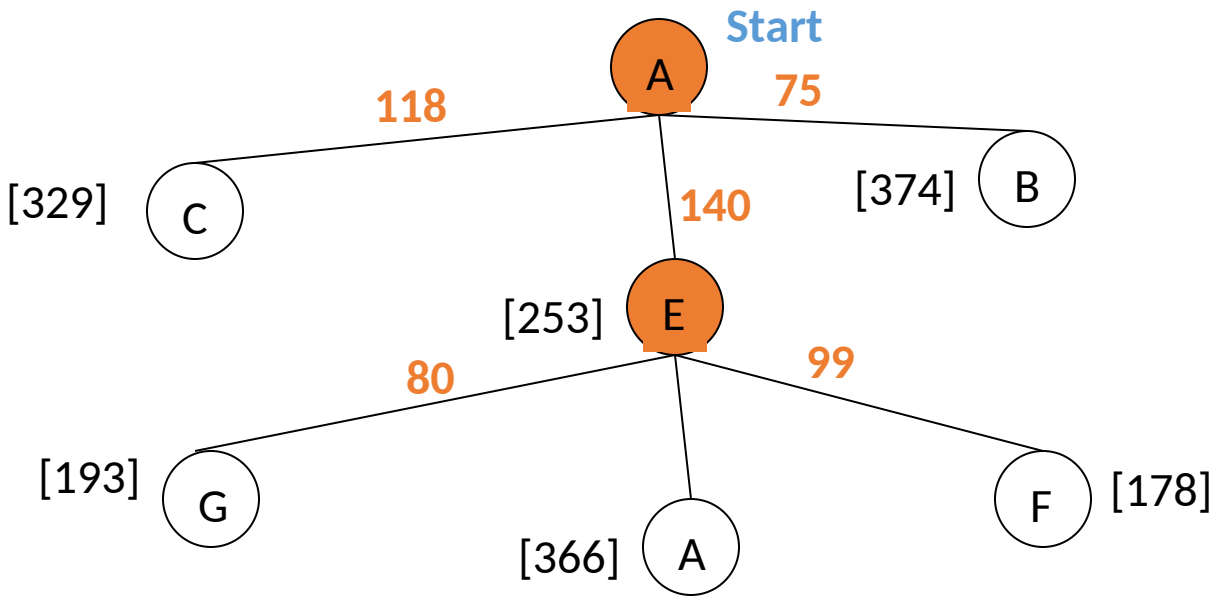
Greedy Search: Tree Search



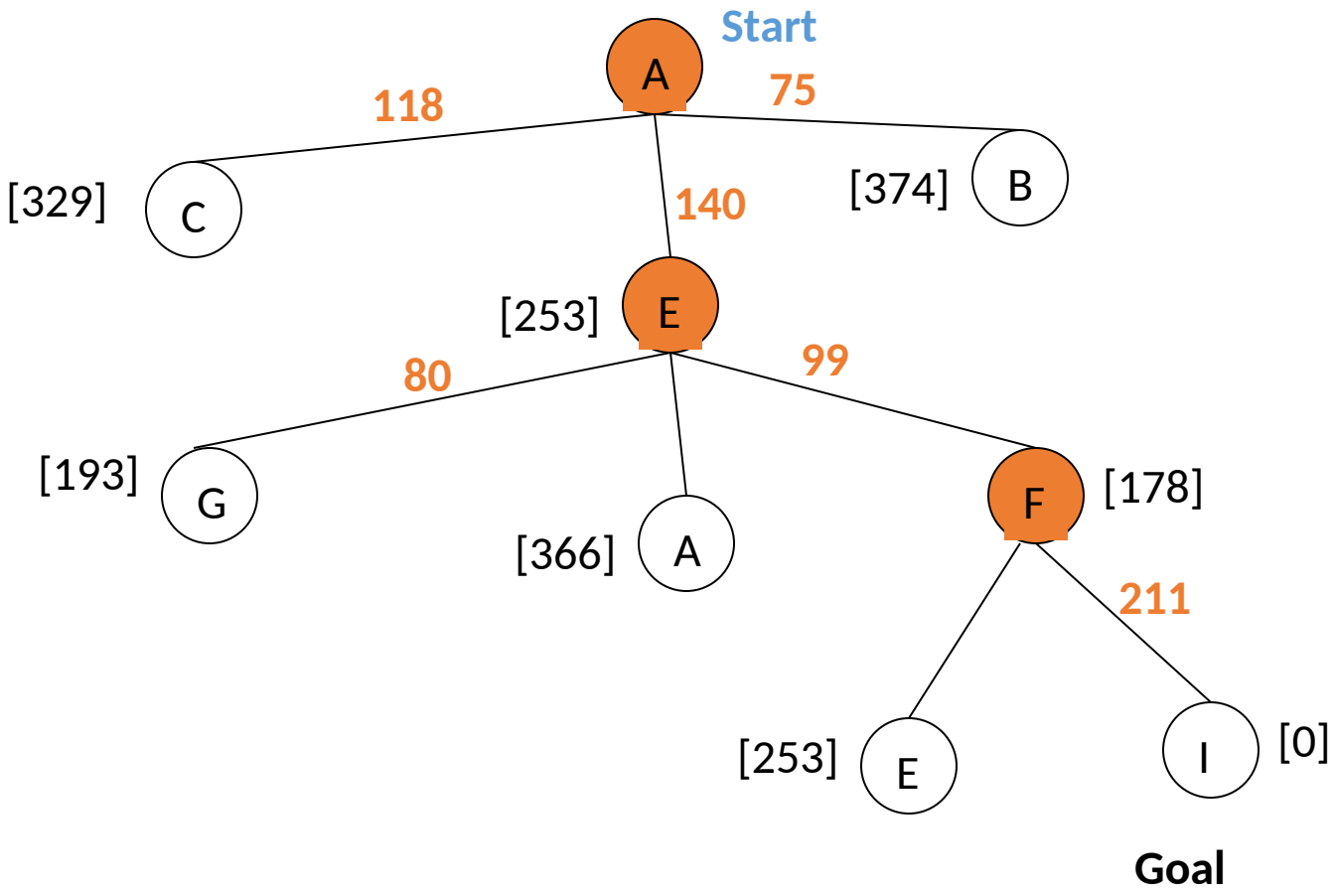
Greedy Search: Tree Search



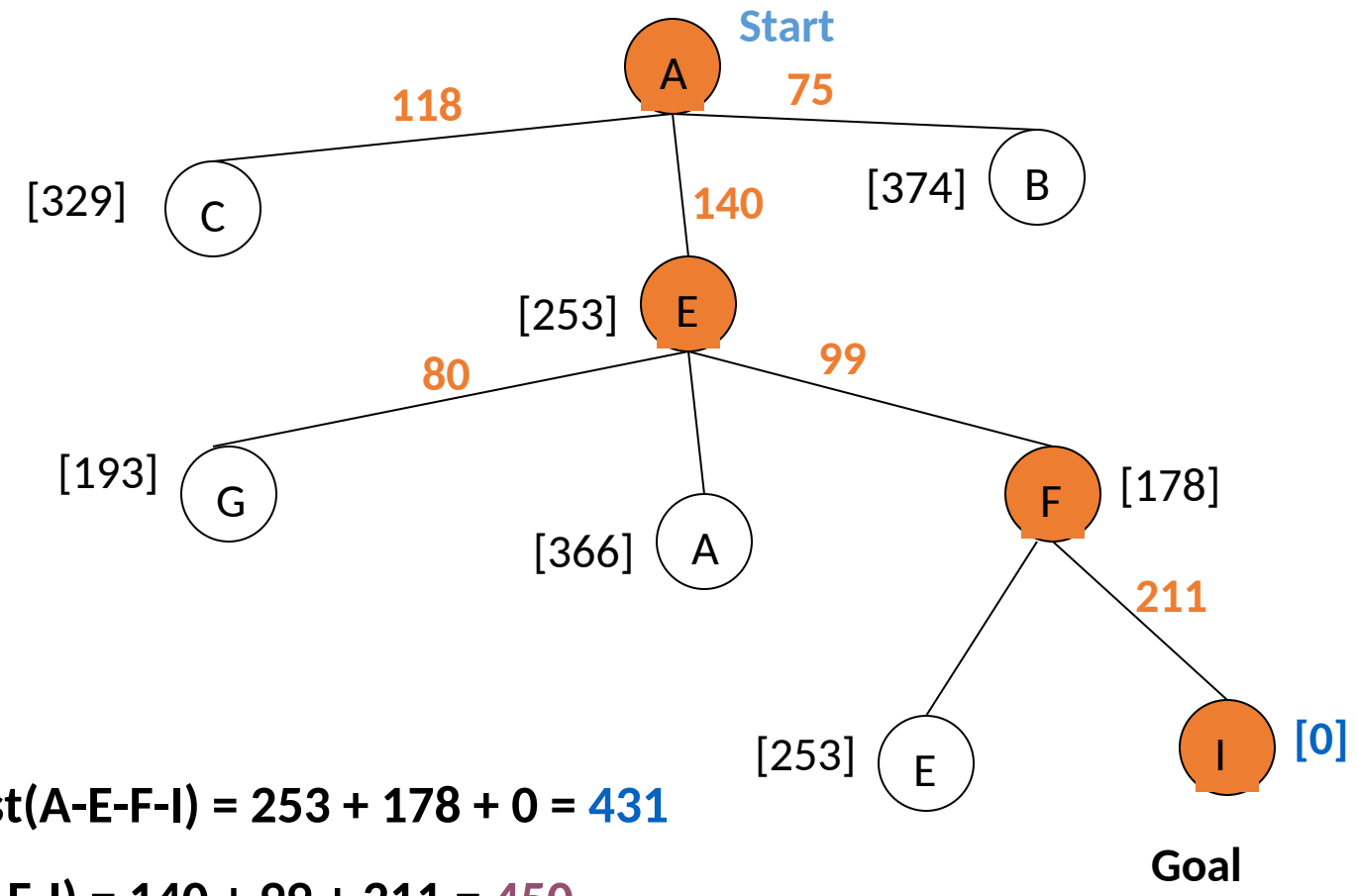
Greedy Search: Tree Search



Greedy Search: Tree Search



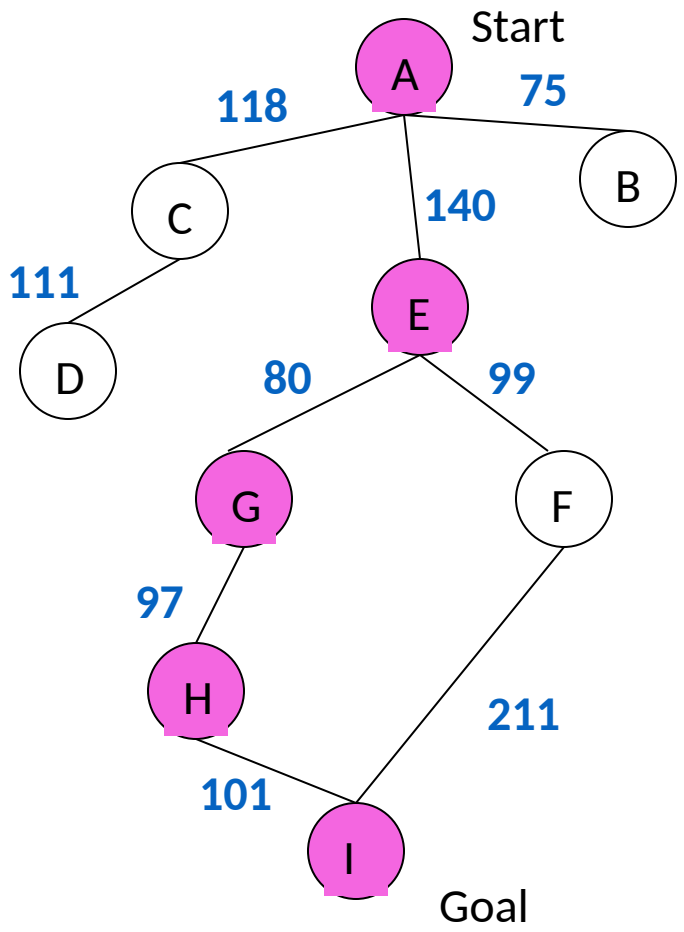
Greedy Search: Tree Search



Path cost(A-E-F-I) = 253 + 178 + 0 = 431

dist(A-E-F-I) = 140 + 99 + 211 = 450

Greedy Search: Optimal?



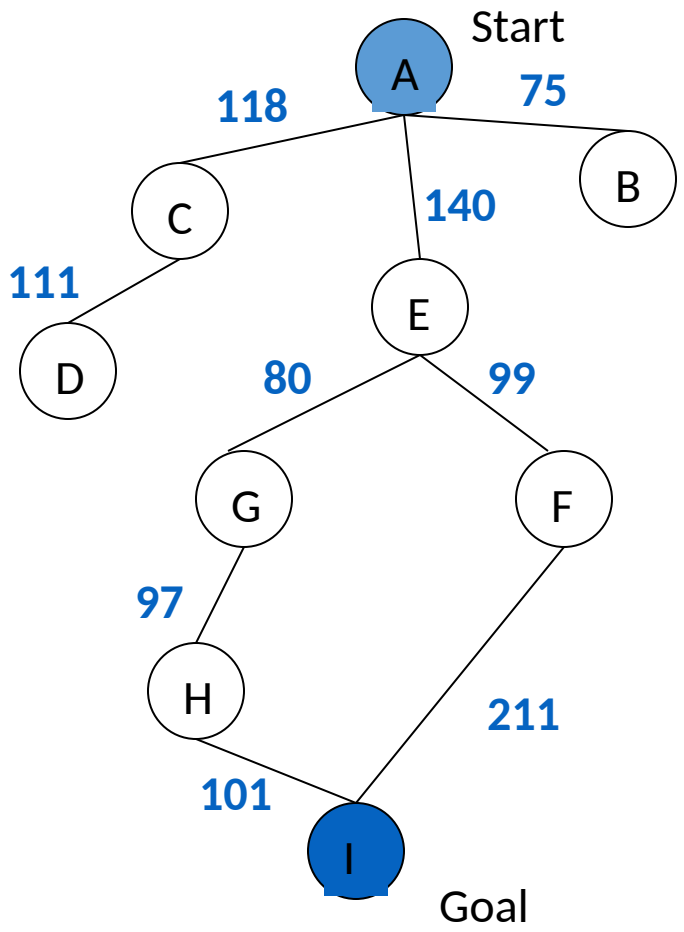
State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

Not optimal!

f(n) = h (n) **= straight-line distance heuristic**

dist(A-E-G-H-I) =140+80+97+101=418

Greedy Search: Complete ?



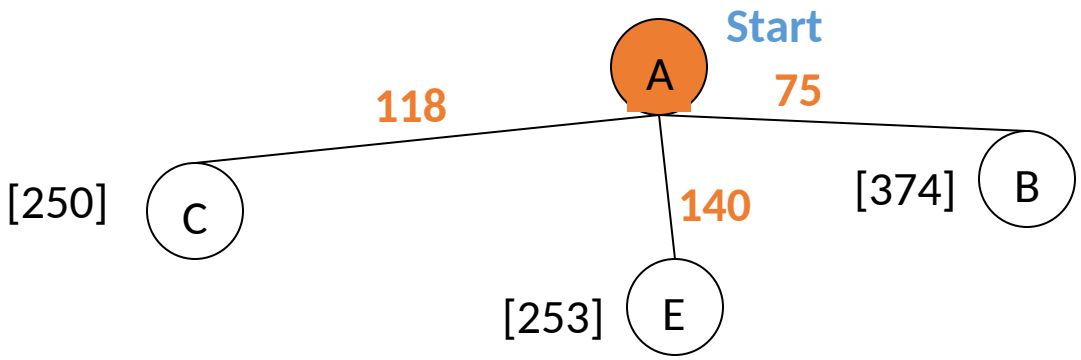
State	Heuristic: h(n)
A	366
B	374
** C	**250**
D	244
E	253
F	178
G	193
H	98
I	0

***f(n) = h (n)* = straight-line distance heuristic**

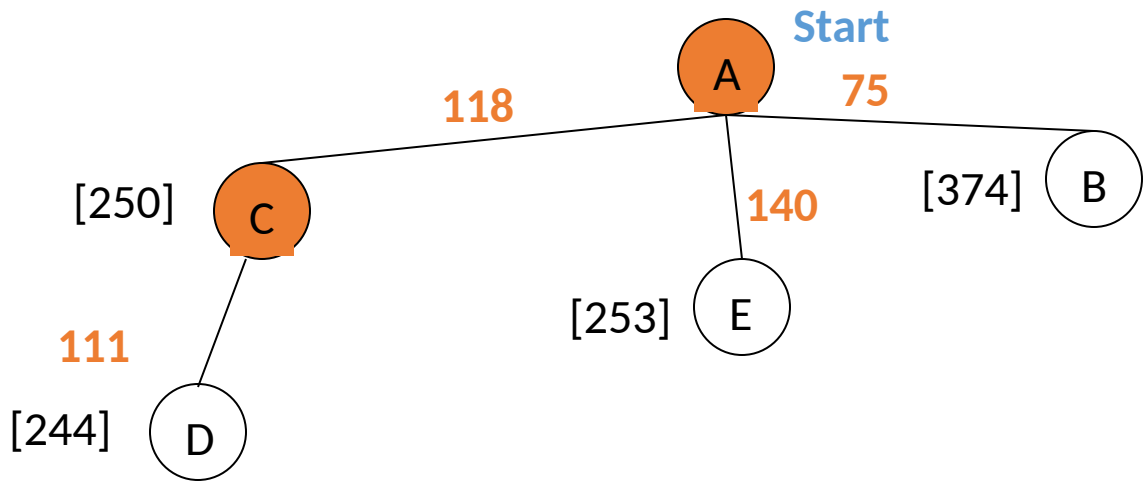
Greedy Search: Tree Search



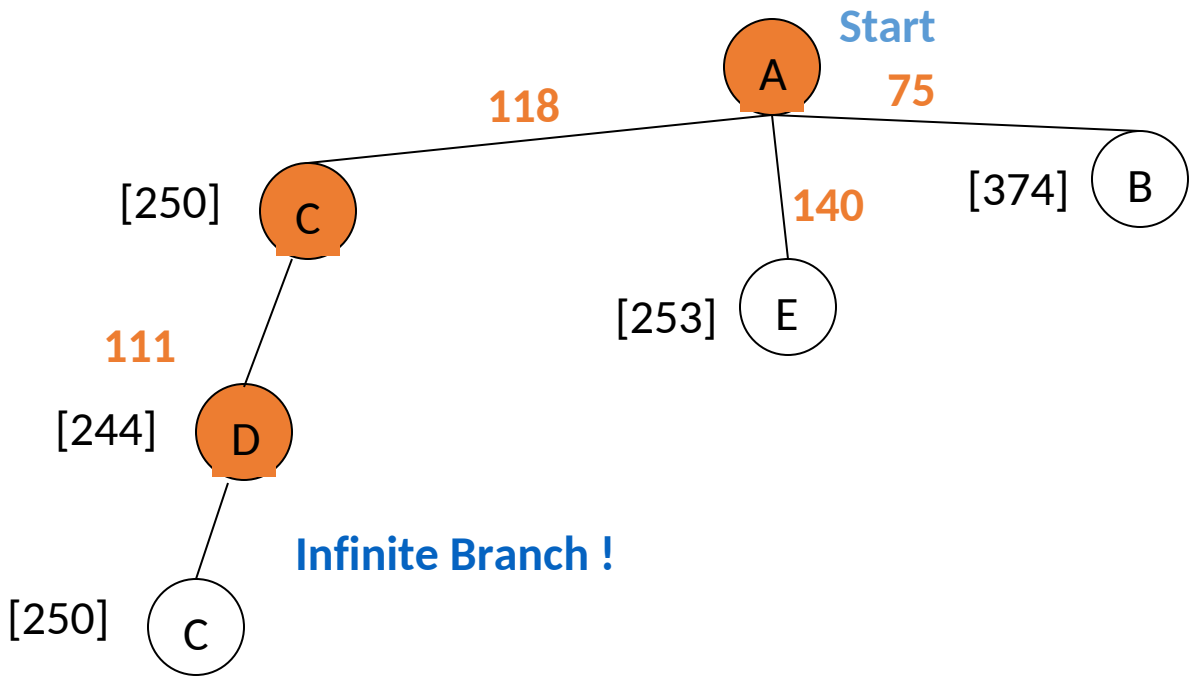
Greedy Search: Tree Search



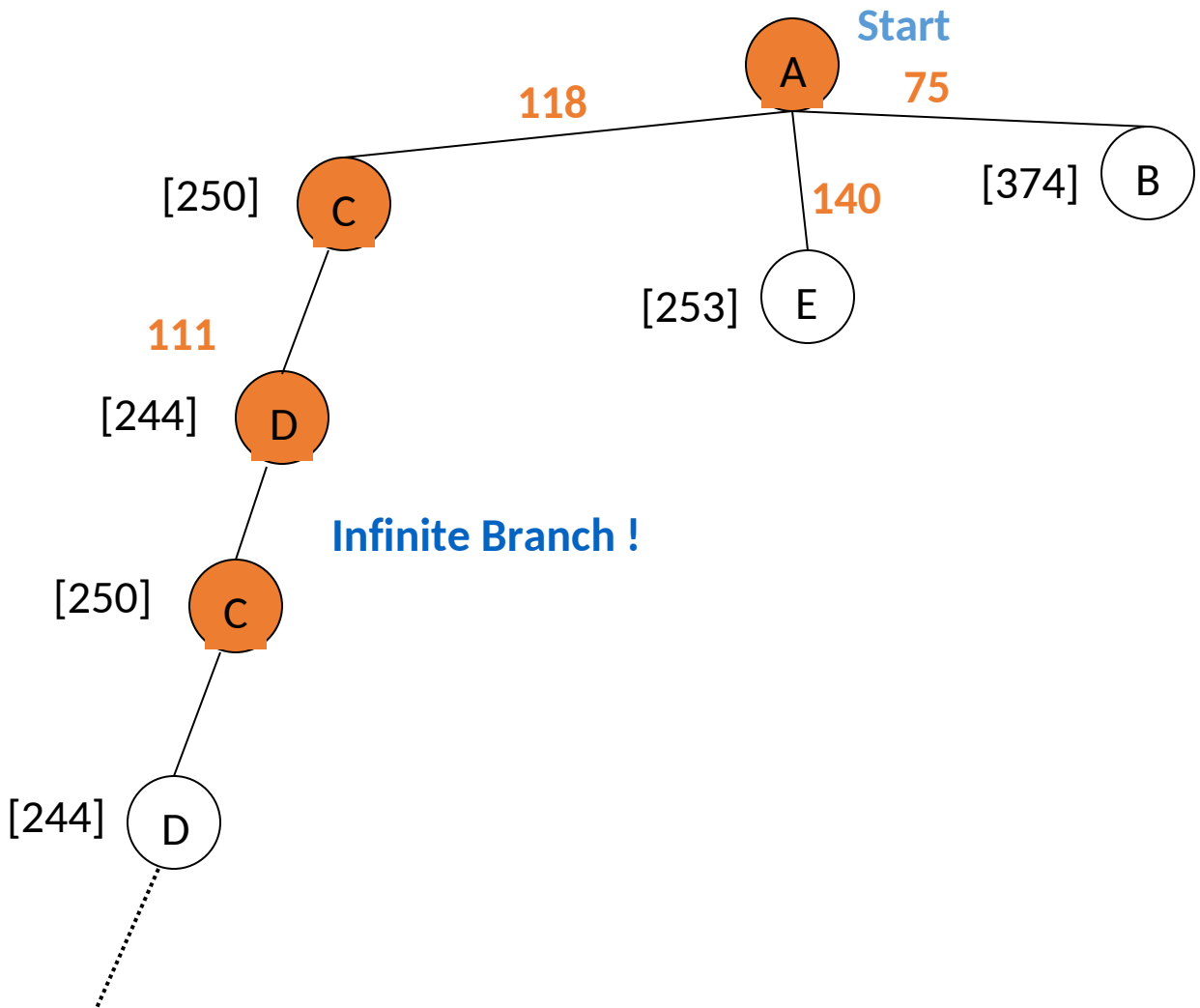
Greedy Search: Tree Search



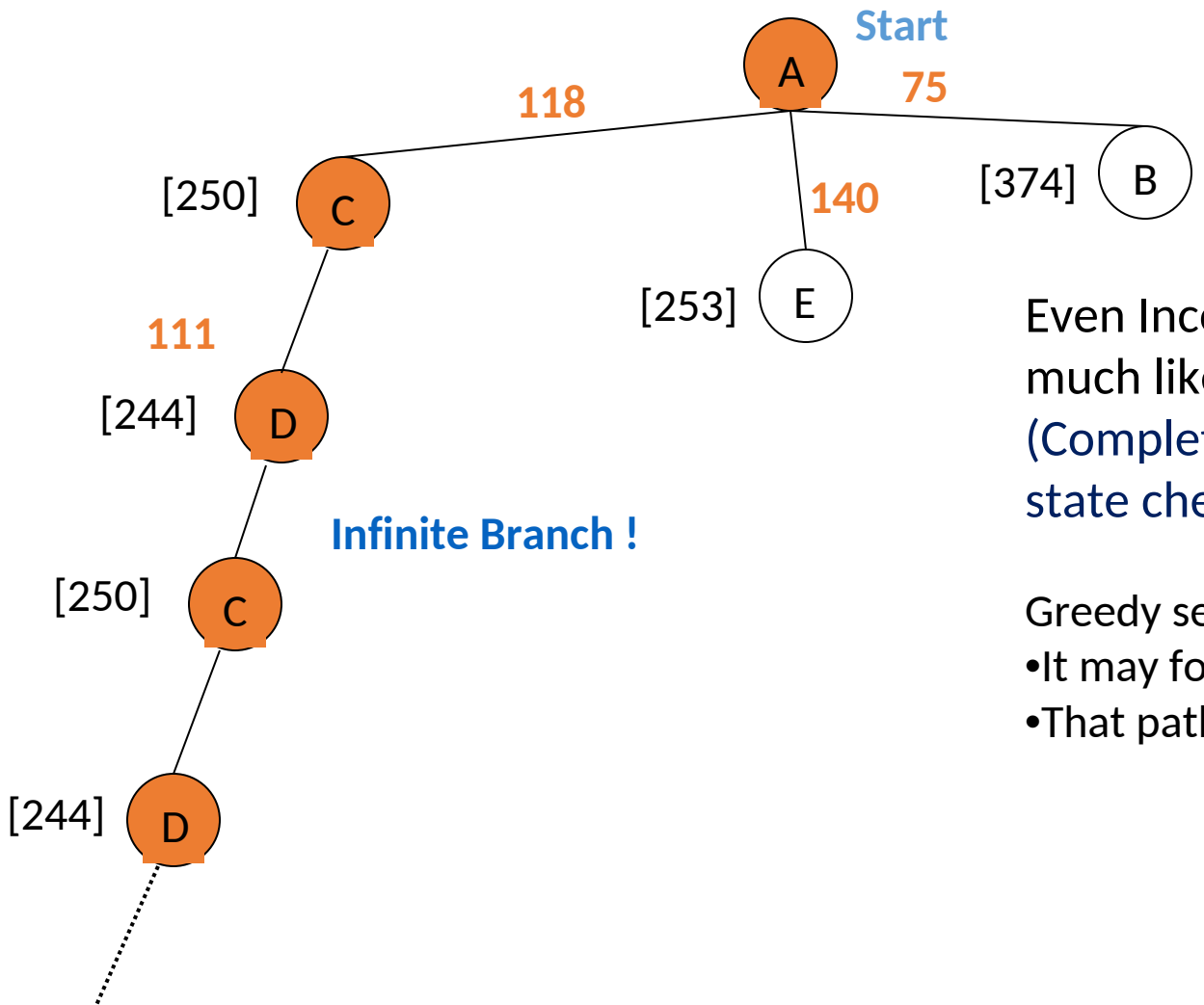
Greedy Search: Tree Search



Greedy Search: Tree Search



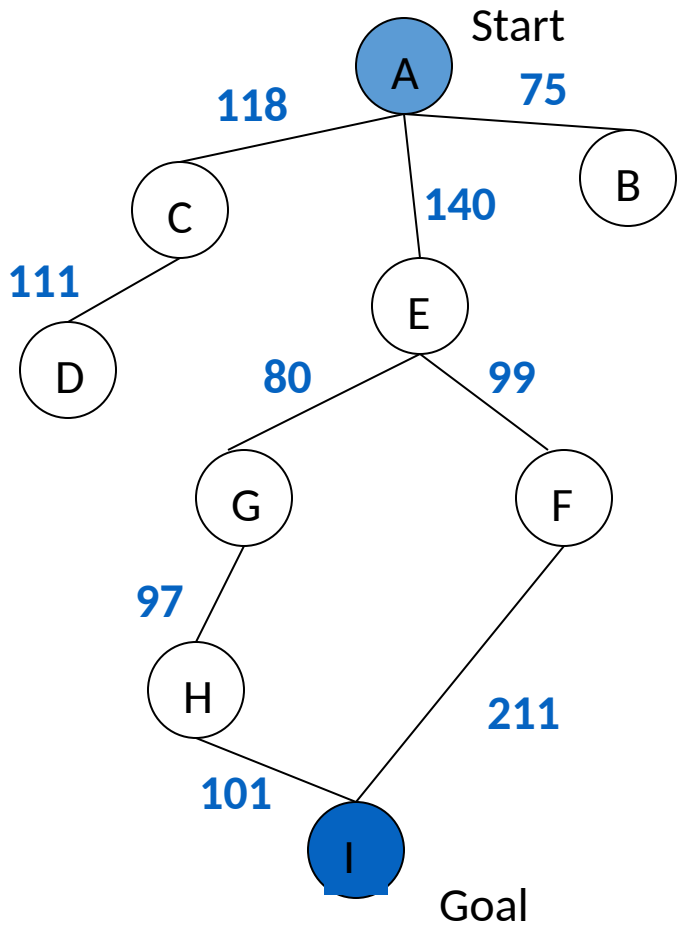
Greedy Search: Tree Search



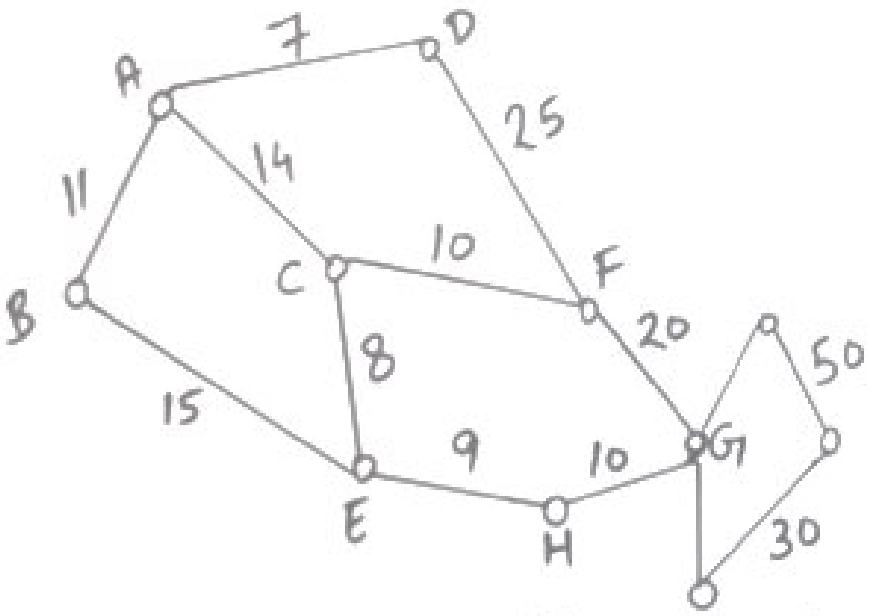
Even Incomplete in finite state space much like depth first search
(Complete in finite space with repeated-state checking)

Greedy search can fail because:
- It may follow a **path that always looks promising**
- That path may lead to an **infinite branch or loop**

Greedy Search: Time and Space Complexity ?



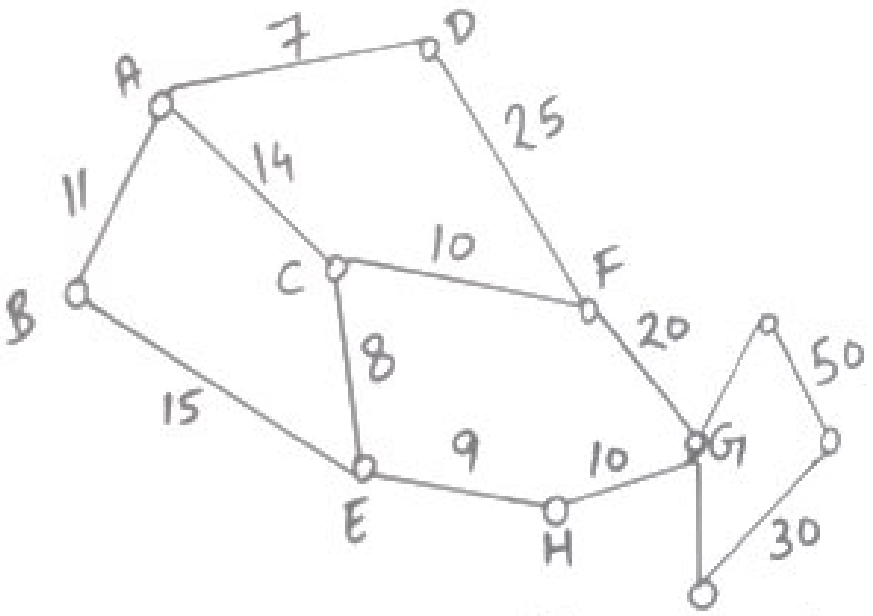
- Greedy search is not optimal.
- Greedy search is not complete
- In the worst case, the Time and Space Complexity of Greedy Search are both $O(b^m)$
- Where b is the branching factor and m the maximum path length



Straight line dist.

A → G1 = 40
B → G1 = 32
C → G1 = 25
D → G1 = 35
E → G1 = 19
F → G1 = 17
H → G1 = 10
G1 → G1 = 0

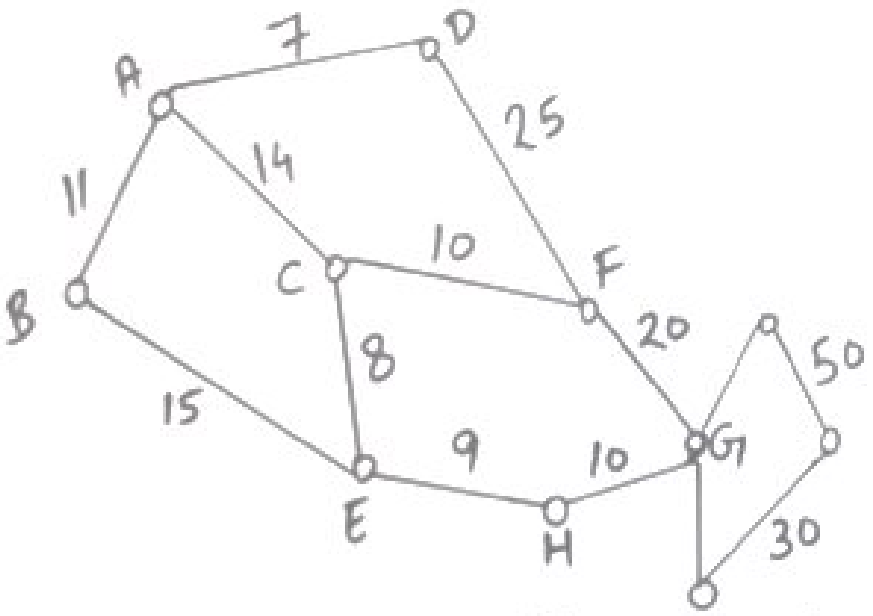
	Frontier (Greedy)	Expand	Explored
1	(A,40)		Empty



Straight line dist.

A → G1 = 40
B → G1 = 32
C → G1 = 25
D → G1 = 35
E → G1 = 19
F → G1 = 17
H → G1 = 10
G1 → G1 = 0

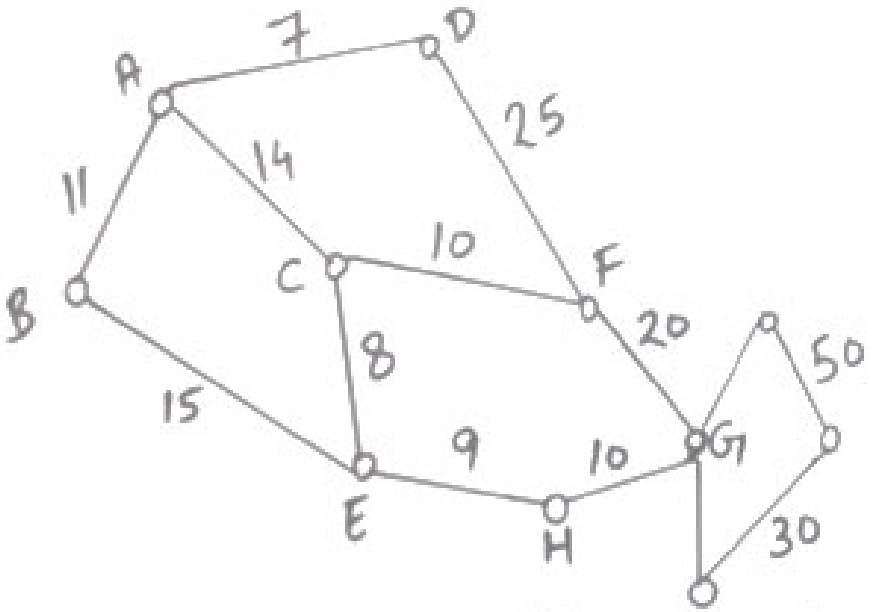
	Frontier (Greedy)	Expand	Explored
1	(A,40)	A	Empty



Straight line dist.

A → G1 = 40
B → G1 = 32
C → G1 = 25
D → G1 = 35
E → G1 = 19
F → G1 = 17
H → G1 = 10
G1 → G1 = 0

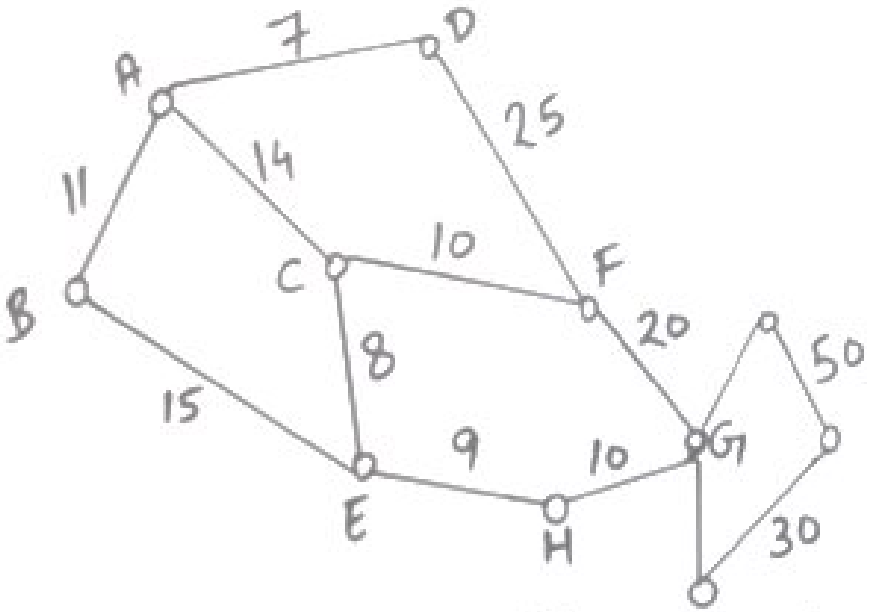
	Frontier (Greedy)	Expand	Explored
1	(A,40)	A	Empty
2	(A-B,32)(A-C,25)(A-D,35)		A



Straight line dist.

A → G1 = 40
B → G1 = 32
C → G1 = 25
D → G1 = 35
E → G1 = 19
F → G1 = 17
H → G1 = 10
G1 → G1 = 0

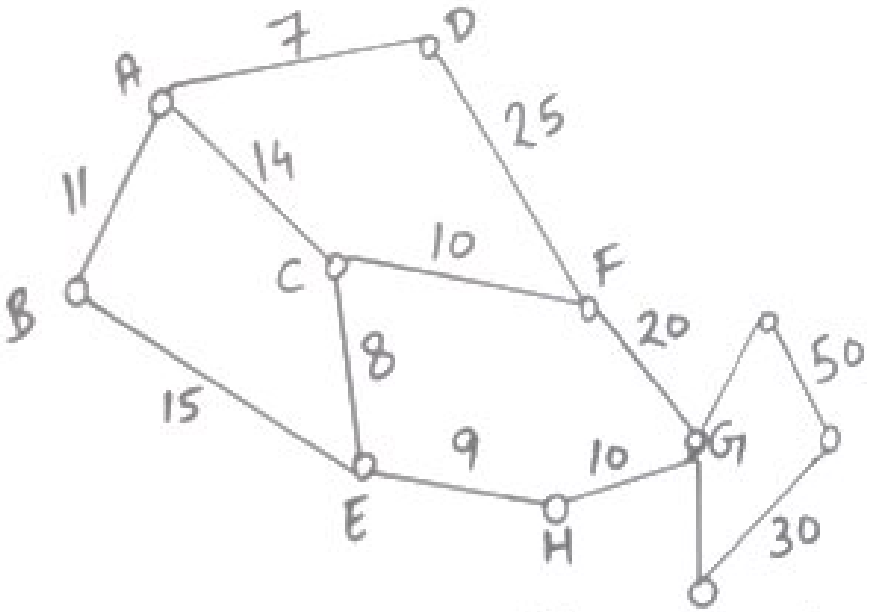
	Frontier (Greedy)	Expand	Explored
1	(A,40)	A	Empty
2	(A-B,32)(A-C,25)(A-D,35)	C	A



Straight line dist.

A → G1 = 40
B → G1 = 32
C → G1 = 25
D → G1 = 35
E → G1 = 19
F → G1 = 17
H → G1 = 10
G1 → G1 = 0

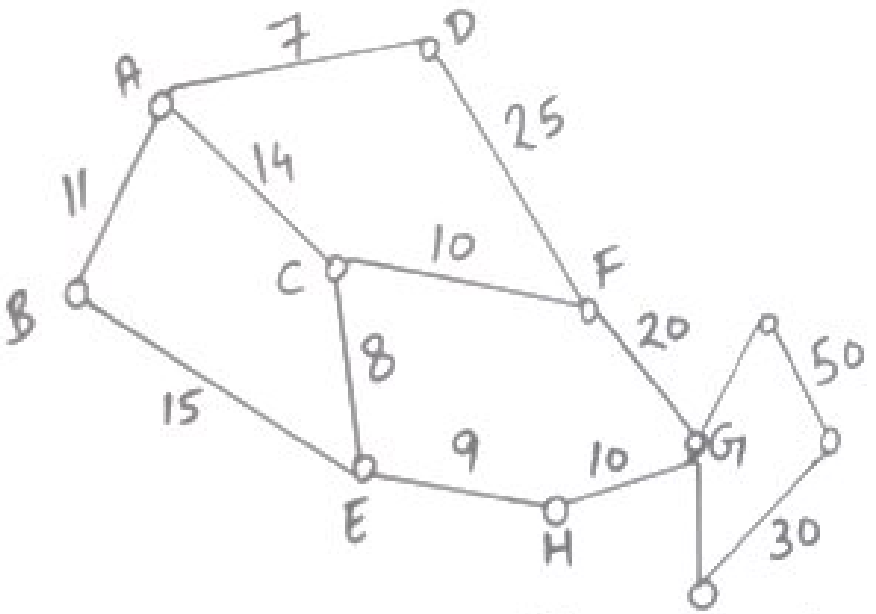
	Frontier (Greedy)	Expand	Explored
1	(A,40)	A	Empty
2	(A-B,32)(A-C,25)(A-D,35)	C	A
3	(A-B,32)(A-D,35) (A-C-E,19)(A-C-F,17)		A,C



Straight line dist.

A → G1 = 40
B → G1 = 32
C → G1 = 25
D → G1 = 35
E → G1 = 19
F → G1 = 17
H → G1 = 10
G1 → G1 = 0

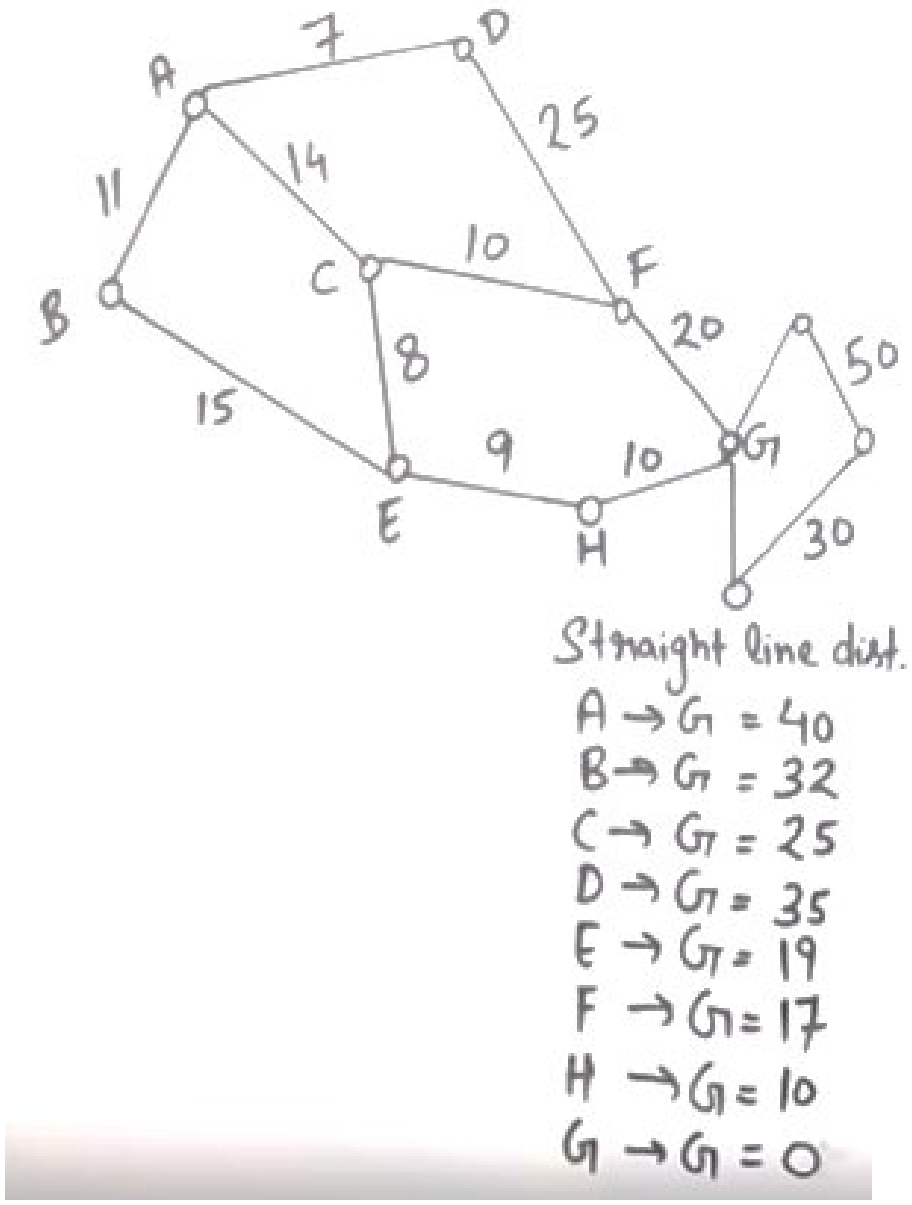
	Frontier (Greedy)	Expand	Explored
1	(A,40)	A	Empty
2	(A-B,32)(A-C,25)(A-D,35)	C	A
3	(A-B,32)(A-D,35) (A-C-E,19)(A-C-F,17)	F	A,C



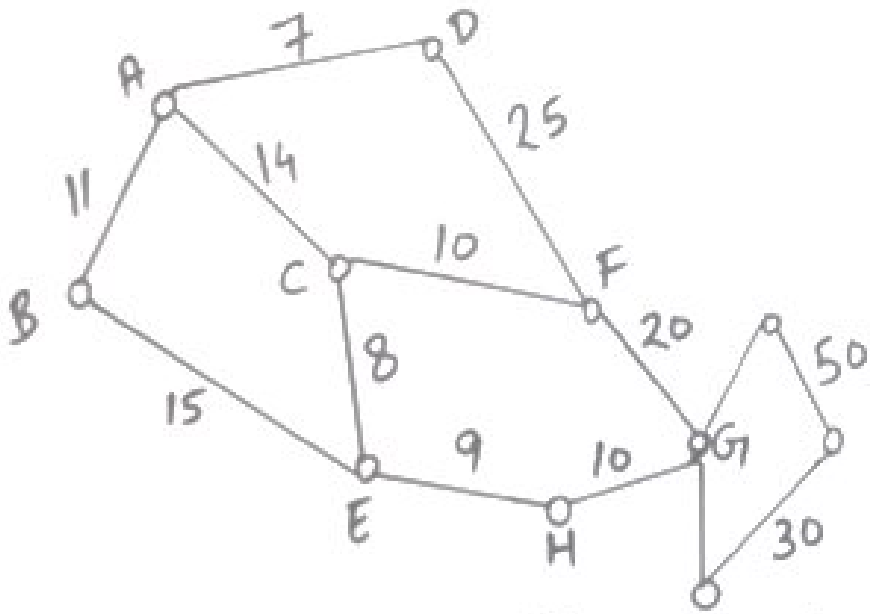
Straight line dist.

A → G = 40
B → G = 32
C → G = 25
D → G = 35
E → G = 19
F → G = 17
H → G = 10
G → G = 0

	Frontier (Greedy)	Expand	Explored
1	(A,40)	A	Empty
2	(A-B,32)(A-C,25)(A-D,35)	C	A
3	(A-B,32)(A-D,35) (A-C-E,19)(A-C-F,17)	F	A,C
4	(A-B,32)(A-D,35) (A-C-E,19)(A-C-F-G,0)		A,C,F



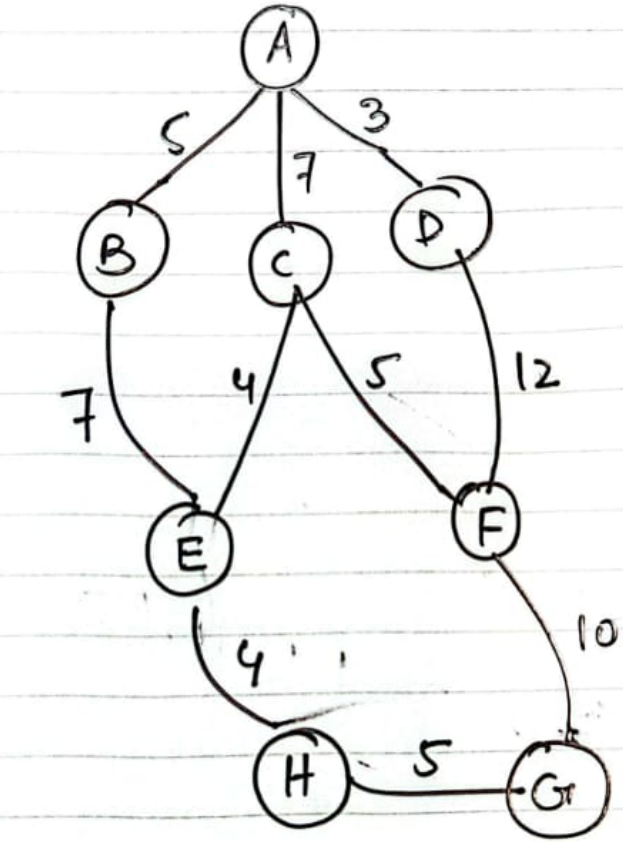
	Frontier (Greedy)	Expand	Explored
1	(A,40)	A	Empty
2	(A-B,32)(A-C,25)(A-D,35)	C	A
3	(A-B,32)(A-D,35) (A-C-E,19)(A-C-F,17)	F	A,C
4	(A-B,32)(A-D,35) (A-C-E,19)(A-C-F-G,0)	G	A,C,F



Straight line dist.

A → G = 40
B → G = 32
C → G = 25
D → G = 35
E → G = 19
F → G = 17
H → G = 10
G → G = 0

	Frontier (Greedy)	Expand	Explored
1	(A,40)	A	Empty
2	(A-B,32)(A-C,25)(A-D,35)	C	A
3	(A-B,32)(A-D,35) (A-C-E,19)(A-C-F,17)	F	A,C
4	(A-B,32)(A-D,35) (A-C-E,19)(A-C-F-G,0)	G	A,C,F
5	(A-C-F-G,0) Goal Found		



h(n)

A = 20

B = 16

C = 12

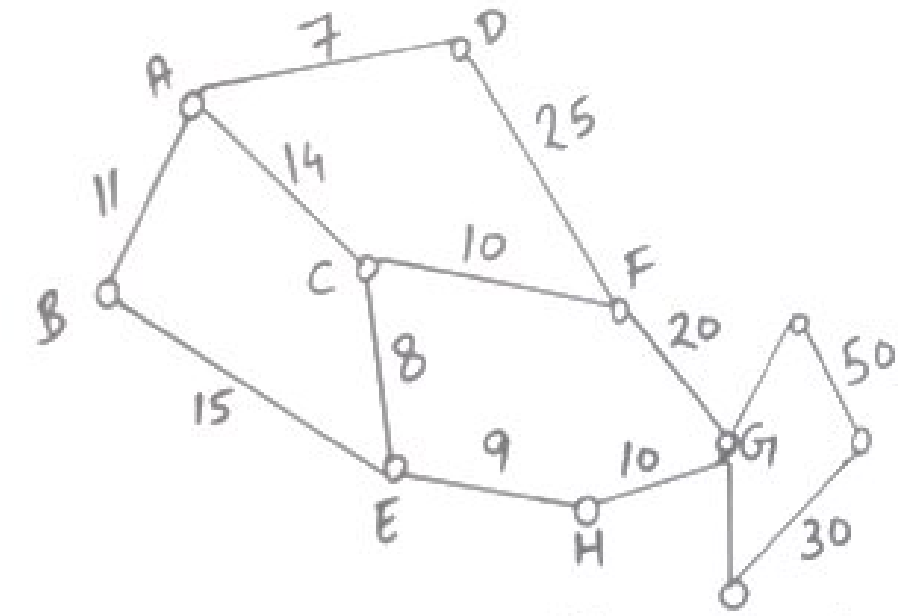
D = 17

E = 9

F = 8

H = 10

Gr = 0



	Frontier (Greedy)	Expand	Explored
1	(A,40)	A	Empty
2	(A-C,25) (A-B,32)(A-D,35)	C	A
3	(A-B,32)(A-D,35)(A-C-F,17)(A-C-E,19)	F	A,C
4	(A-B,32)(A-D,35)(A-C-F-G,0)(A-C-E,19)	G	A,C,F
5	(A-C-F-G) Goal Found		A,C,F,G

	Frontier (UCS)	Expand	Explored
1	(A,0)	A	
2	(A-B,11)(A-C,14)(A-D,7)	D	A
3	(A-B,11)(A-C,14)(A-D-F,32)	B	A,D
4	(A-B-E,26)(A-C,14)(A-D-F,32)	C	A,D,B
5	(A-B-E,26)(A-C-F,24)(A-C-E,22)(A-D-F,32)	E	A,D,B,C
6	(A-B-E,26)(A-C-F,24)(A-C-E-H,31)(A-D-F,32)	F	A,D,B,C,E
7	(A-B-E,26)(A-C-F-G,44)(A-C-E-H,31)(A-D-F,32)	E	A,D,B,C,E,F
8	~~(A-B-E,26)~~(A-C-F-G,44)(A-C-E-H,31)(A-D-F,32)	F	
9	~~(A-B-E,26)~~(A-C-F-G,44)(A-C-E-H,31)~~(A-D-F,32)~~	H	
10	A-C-E-H-G,41 Goal Found		

Heuristics

- Problem:
  - A heuristic is only an informed guess of the next step to be taken in solving a problem.
  - It is often based on experience or intuition.
  - Heuristics use limited information, such as knowledge of the present situation or descriptions of states currently on the open list.
  - They are not always able to predict the exact behavior of the state space farther along in the search.
  - A heuristic can lead a search algorithm to a suboptimal solution or fail to find any solution at all. This is an **inherent limitation** of heuristic search.